

Applied Machine Learning
Data-Driven Crop Yield Forecasting: Analyzing Indian Agricultural Data (2010-2017)

RUTHUGUA NATALIE KARIMI

Abstract

This document explores data-driven crop products, machine learning, and advanced analytical techniques on agricultural data from 2010 to 2017 in India. It undergoes several studies that use monitored learning models, which include random forests, SVMs, LSTMs, and hybrid approaches to predict the dividend. The work emphasizes the importance of extensive data preparation, significant future factors such as rainfall, earth properties, and management practices, and the need for region-specific modeling strategies. Conclusions suggest that the artist's contingent, strong functional technique and integration of environmental parameters increase predictive accuracy, contributing to informed decisions and food safety and stability initiatives in Indian agriculture.

Table of Contents

.....	1
<i>Abstract</i>	2
<i>Part A:</i>	4
1. Introduction.....	4
2. Aim & Objective	4
3. Related Works.....	5
<i>PART B:</i>	14
<i>Dataset Preparation</i>	14
<i>Exploratory Data Analysis</i>	17
Key Insights from the Plots:.....	20
Data Preprocessing	30
<i>Model implementation</i>	34
<i>Decision Tree Regressor Implementation</i>	36
<i>Random Forest Regressor.</i>	39
<i>Gradient Boosting Regressor</i>	41
<i>XGBoost Regressor Model</i>	44
Model Validation and Analysis & Recommendations.....	47
Conclusion	49

Part A:

1. Introduction

Agriculture has a central place in India, which supports more than 70% of the population of the country living in rural areas and acts as their primary business. India is the second-largest global manufacturer of wheat, fruits, milk, tea, and fruit, with an important basis for food safety and economic stability. The region reduces efforts to promote sustainable development significantly, adapt to climate change, and meet the needs of the growing population. Data for crop production plays an important role in revealing trends in crop patterns, such as changes in high-value crops such as vegetables and fruits from coarse grain, and helps with a resource plan for better productivity and stability. The analysis of such data helps to identify regional differences and support effective decision-making for farmers, decision-makers, and researchers, leading to better strategy in crop diversification and resource management. Important drivers for agricultural productivity include water, proper soil and climatic conditions, and the availability of quality seeds, fertilizers, credits, and infrastructure for irrigation, while the government's policy and technological innovations have continued the results. However, Indian agriculture is facing sufficient challenges, especially since there is an urgent need for the use of technology and efficient use of resources to ensure climate change, water loss, decline, and long-term stability. (Pattanayak, 2017)

2. Aim & Objective

Aim:

The aim of this project is to analyze crop production data for all Indian states from 2010 to 2017, identify trends in crop yield, acreage, and seasonal patterns, and propose methods for improving crop diversification and sustainability.

Objectives:

- To explore the crop production data for Indian states over the years 2010-2017.
- To identify significant patterns and regional differences in crop yield and acreage.
- To evaluate multiple models for predicting crop yields based on historical data.

3. Related Works

- **Investigating the Linkage between Economic Growth and Environmental Sustainability in India: Do Agriculture and Trade Openness Matter**

The relationship between economic development, environmental sustainability, and agriculture in India is a critical issue, as the country struggles with growing CO₂ emissions and a heavy dependence on fossil fuels. A study by Orhan et al. (2021) examines how CO₂ emissions interact with economic growth, agricultural activities, energy consumption, and trade openness using Indian data from 1965 to 2019. This summary reflects their findings and explores methodologies, policy implications, and a detailed evaluation. The key research questions include: What is the connection between agriculture, energy consumption, trade openness, and CO₂ emissions? How do these factors affect environmental sustainability in India? The study suggests that agriculture and energy consumption are major contributors to CO₂ emissions, while trade openness has a minor impact. To analyze these relationships, the researchers use unit root tests for data stationarity, cointegration tests to explore long-term connections, and the wavelet coherence test, which identifies patterns at different time scales. The Gradual Shift Causality Test can still find causation even when data includes structural breaks. This gives us strong information about how variables interact.

The wavelet consistency indicates that CO₂ emissions, economic growth, and agriculture are all stronger and positively linked to moderate and long-lasting. CO₂ emissions and energy consumption are positively correlated, reflecting India's coal addiction. Shift tests gradually show that energy consumption and economic expansion drive emissions, but agriculture, the response, suggests loops. The opening business has a small and inconsistent effect on the emissions. Data is often achieved by using natural log changes in the preparation stages. The study suggests that renewable energy and existing agricultural methods should switch when affecting production results. When it comes to detecting complex links between agriculture and the environment, the wavelet consistency method is better than time chain methods because it can show how the link changes over time. But it cannot come up with predictions or preference assessments or be able to do any machine learning. In the future, machine learning can be used to make return estimates more accurate and to identify hidden patterns that will do wavelength analysis and more accurate forecasts. The government must do something about India's agriculture and environmental policy. Research plants trees for agriculture that is organic and has minimal emissions, and must expand solar, wind, and hydrometer. Supporting renewable energy in the private sector helps the environment and makes people strong.

These rules have an impact on the methods for agriculture, the use of technology, and the use of resources, all of which are important to provide accurate predictions on future returns. In conclusion, Orhan et al. assert that agriculture and energy consumption are the main drivers of

environmental degradation in India, a finding supported by the time-frequency analysis. Business openness is found to be a minor factor. Proof emphasizes the need for policy that promotes stability in both energy and agricultural sectors. Future research should consider using more indicators for environmental decline and integrate the machine learning model for better future analysis. Political decision-makers should support permanent agriculture to integrate agriculture and environmental policy, use green technologies, and improve productivity and environmental health. (Orhan et al., 2021)

- **The Impact of Groundwater Depletion on Agricultural Production in India**

The document examines the use of profit-based crop models, as well as the use of district levels with a fixed power panel region to assess the results of groundwater deficiency on agricultural productivity in India. These models provide valuable insights into the impact of environmental factors on crops, crop area, and production. When it comes to function, the document provides a detailed interpretation of the regression model, which highlights the use of fixed effects of the district and the year, which decides to control without any factor. Westers also perform a strong test by focusing on districts with sufficient groundwater data and testing alternative models' specifications to ensure reliability. However, they accept boundaries, such as problems with data accuracy, potential bias in district-level reporting, and simplification of groundwater data using mean values, which cannot occupy local variations. Writers highlight the importance of using technology, such as drip watering and laser leveling, and encouragement for farmers, but do not specify functional strategies for future research that are outside the approaches used in the studies. Finally, while the paper provides intensive analysis through regression and economic modeling. Future research recommendations focus on policy intervention and technology instead of the inclusion of new forecast modeling methods. (Bhattarai et al., 2021)

- **Machine Learning Models for Crop Yield Prediction in Indian Agriculture**

Agricultural forecasting is important to ensure food security and economic stability in India and globally. This literature review is the center that is at work by Jhajharia et al. Check which models are effective for forecasts, how regional and seasonal factors are included in the predictions of returns, pre-probing methods that increase the model performance, and whether an impressive future-centered factor is assessed for accurate forecasts. The main issues are how the environment and historical data are used in the predictions about crops and how to evaluate machine learning methods.

Jaznjaria et al. analyzed the study crop data for larger crops in Rajasthan in 33 districts from 1997 to 2019. The dataset was cleaned by removing the incomplete items, by coding the range of variables (e.g., district and soil), and by using the standards to standardize the functions. Dummy variable coding was used to convert the classified data to machine education formats. Important predictions were identified for crop means, agricultural sector, production level, soil type, and seasonal rainfall, which emphasizes the importance of agricultural environment data. To address

regional and seasonal variations, synchronized rain data with specific crop months, coded district-specific soil types, and eliminated items with insufficient data to reduce errors. The authors used many models, including random forest, supporting vector machine (SVM), gradient descent, long short-term memory (LSTM), and LASSO regression. The model comparison was based on the screen measurements, such as R, Root Mean Square Error (RMSE), and average absolute error (MAE). The results showed that random forest got the best performance (RMSE 0.035, MAE 0.025) when all facilities were assessed. It outperformed SVM, Lasso Regression, and LSTM, with the latter underperforming due to data constraints rather than the model's limitations. The authors conclude that ensemble machine learning methods, especially Random Forest, offer significant effectiveness in the Indian agricultural context. However, they suggest that deep learning models, such as LSTM, could improve with access to larger datasets. For future research, integrating remote sensing data and expanding the dataset are recommended to enhance predictive accuracy and broaden the application of the models to other agricultural challenges. (Jhajharia et al., 2023)

- **Machine Learning-Based Crop Yield Prediction in South India: A Critical Review**

India and the world depend on agricultural forecasting for food security and economic stability. Nikhil et al. (2024) examined machine learning and deep learning methods for crop output prediction in South India. The study looks at which models work best for predicting yields for crops such as rice, sorghum, cotton, sugarcane, and rabi. The study suggests that machine learning techniques, especially wood-based models, can increase the accuracy of predictions of crops in the southern Indian context significantly. Research collects data from different sources, including meteorological and soil data for more than 15 years, ensuring diversity and coverage throughout the region. Many machine learning models, including random forests, LSTMs, and gradients that increase, were used to evaluate the prediction of crop dividends. The results show that tree-based models, as several trees performed best.

These models achieved high RAP values; the extra trees reached 0.9615, indicating strong future power and a low error rate. In contrast, linear models were reduced due to the inability to capture complex conditions that lie in agricultural data. The study emphasizes the important role of using non-LED models to improve predictions. It recommends several fields for future research, including the use of granulated cosmic data, improvement of data quality, and detection of hybrid models along with tree-based techniques. By increasing the interpretation of the model and improving data collection and processing, the study emphasizes that the machine learning technique can lead to more reliable, scalable, and precise agricultural forecast models. (Nikhil et al., 2024)

- **Crop Yield Prediction Models: Approaches, Challenges, and Insights**

Accurate crop output forecasts benefit agriculture, food security, and policy. Some models predict agricultural yields using crop growth, while others use statistics and machine learning. This

literature analysis examines 101 Indian and global papers on methodology, important aspects, obstacles, and integration of modeling methodologies. Most important: What agricultural forecasting models are used? How to address seasonal and geographic differences? Most efficient preprocessing features and methods? What problems has research faced while contrasting and integrating modeling approaches? The concept is that process-based and data-driven models improve forecast accuracy and reliability. Scopus was used to review 2009–2019 process-based or data-driven agricultural production prediction literature. This comprehensive study examined field, farm, and regional study models, input data, and integration approaches. From 109 studies, 101—including hybrid models were chosen for further study. Biophysical process-based models like APSIM, DSSAT, and AquaCrop can mimic crop growth at the field size with exact management data (Maestrini et al., 2022). Neural networks, support vector machines, linear and non-linear regressions, random forests, and the US USDA-NASS are employed more in countries with abundant datasets (Maya Gopal & Bhargavi, 2019).

We predict regional and seasonal changes using irrigation, fertilizer, and meteorological data, including rainfall, temperature, and radiation. Farms use management data more than regional meteorological data. To account for regional and national farm management differences, Jeong et al. (2016) and others incorporated fertilizer consumption. Cleaning, feature engineering (NDVI from reflectance data), and pretransfer functions for missing soil parameter estimates are effective preprocessing procedures. Crop models and domain expertise affect feature selection, which affects model performance. Agricultural output is predicted by weather, soil, management, and remote sensing indices like NDVI and EVI. The "farm factor," farmers' knowledge and actions, is also crucial on farms, according to Sylvester-Bradley et al. (2019). The comparative study demonstrates there is no best model; it depends on circumstances and situation. Large datasets demand deep neural networks, whereas smaller, more detailed datasets can employ simpler models. While accuracy varies with dataset, R^2 and RMSE are used to assess performance. Metamodels to reduce computational complexity, data-driven models to estimate missing inputs for crop models, and crop models to supply features for data-driven models are the three basic integration methodologies.

The benefits of metamodels and transfer learning can be combined. Lack of large, reliable datasets, data quality concerns (such as noise or bias in self-reported yields), and process-based models' inability to account for all yield-reducing factors, such as sickness and pests, are issues. These solutions combine methodologies and employ better data platforms. Maya Gopal and Bhargavi (2019) examined crop yield prediction models ANN, SVR, k-NN, and Random Forest in Tamil Nadu, India. The literature advocates process-based and data-driven agricultural production prediction models. All have pros and cons. Metamodels and transfer learning increase prediction accuracy and scalability. Poor data quality and oversimplified models persist, but future studies should enhance integration methods and develop field-usable, open-access agricultural datasets. Domain knowledge and machine learning can improve global farming and policymaking. This improves agricultural diversity and sustainability. (Maestrini et al., 2022)

- **Machine Learning Models and Data-Driven Approaches in Indian Agricultural Forecasting**

According to Pawlak and Kołodziejczak (2020), farming is crucial to India's economy, as over 58% of the population relies on it. This industry must address soil erosion, climate change, and long-term solutions. Data-driven precision agriculture and machine learning (ML) have. Improved crop and resource optimization. This literature review covers recent agricultural forecasting models, data pretreatment, prediction criteria, ML model performance, and weather and location change investigations. This essay answers critical questions like, which machine learning methods estimate agricultural output most often? Can agricultural yield projections account for seasonal and regional changes? How can preprocessing improve model accuracy? Which features and methods perform best? The challenge of projecting agricultural production from past data and how climate and geography affect predictions are also highlighted. Find patterns in 2010–2017 Indian agricultural output statistics to grow more diverse and sustainable crops.

Catarya et al. (2020) and Rajamanickam and Mani (2021) found that most studies used supervised ML models, including SVM, Naive Bayes, LSTM, KNN, neural networks, and ensemble. Kaggle datasets include soil pH, NPK, humidity, rainfall, and temperature. Preprocessing comprises cleaning, feature extraction, and training/test set separation. Popular model evaluations include F1-score, recall, accuracy, and precision. SVM was most accurate (91%), followed by Naive Bayes (87%) and LSTM (83%), according to Reddy et al. Rain, temperature, and soil nitrogen affect estimations. Studies account for regional and seasonal changes using agro-climatic zones and environmental factors (Chaudhari et al., 2018). PCA and learning parameter fine-tuning may improve models (Suchithra & Pai, 2020). Indian models are difficult to use due to soil and weather differences.

The literature implies SVM and NN models can predict agricultural yields. This is especially true with excellent preprocessing and feature selection. Models with regional and seasonal variability benefit from local environmental data. Crop yields depend most on soil nutrients, pH, rainfall, and temperature. Models are evaluated by recall, accuracy, precision, and F1-score. Although promising, the models need more testing in real Indian farming circumstances and data to address regional variations.

Next, validate the models in different Indian agro-climatic zones, integrate environmental and policy components, and expand databases with regional and temporal data. Agronomists, technologists, and policymakers must work together to develop flexible food security and sustainable farming systems in India. (Reddy et al., 2024)

- **Machine Learning Models and Methodologies for Crop Yield Prediction in India**

For agricultural policy, administration, and schemes, the crop production forecast should be accurate. Especially in agriculture in India. This literature review has recently described hybrid time series model studies, which predicted the Indian Rabi crop production using external inputs. Data preparations, important aspects, seasonal and regional ups and downs, and comparison of the model were discussed. The study addresses important questions: How do agricultural forecast models use models? Treatment with seasonal and regional variety? Do caring methods do the best work? Which crop forecast factor is most important? How is the machine learning model compared? Are any strategies and symptoms more effective? How do climate and geography affect crops? Can you clarify problems and solutions? Evaluation uses this matrix. Does anyone know about similar dataset research, and what can we learn?

For studies, India presented annual wheat, sugarcane, and peanut production (kg/hectares) and irrigation area data. This data is of government data from 1950-2019. MAPE and RMSE assessed machine learning and time series models. Make training and test kits, then create tests and registration models. ARIMA, ARIMAX, TDNN, NLSVR, WNN, CNN, RNN, LSTM, and hybrid ArmX models were used. The ADF and BDS stability and non-control tests were required during advance.

The study found that hybrid models, particularly ARIMAX-LSTM, predicted crop yield better than independent ML models. ARIMAX-LSTM enhanced model accuracy by 10.41% and 12.28% over other models and 15.83% and 18.42% over ARIMA by adding area under irrigation as an exogenous variable. Non-linearity and stationarity preprocessing required residual analysis and differencing. Past yields and irrigation area best predicted future yields. A 0.96 correlation showed a considerable association between wheat and sugarcane. Indian states have varying yields because of topography, climate, and irrigation. RMSE- and MAPE-based studies revealed hybrid models can adapt to agricultural industry changes.

India's biggest Rabi crop harvest yields were better estimated by hybrid time series models, particularly ARIMAX-LSTM. Exogenous variables like irrigation and advanced neural network topologies can reveal linear and non-linear agricultural data trends. The research advises against generalizing model performance to hybrid models. Future research should study hybrid architectures and exogenous variables to improve prediction accuracy.

Practical experience emphasizes irrigation infrastructure investments and data-driven farm planning. The research methods and results can inform India's agricultural crop diversification and sustainability policies. These conclusions are supported by irrigation projects in Punjab, Haryana, and Uttar Pradesh, which enhanced wheat yields. (Pandit et al., 2023)

- **IoT-Based Smart Agriculture in India**

The rapid growth of the global population, which is expected to reach 10 billion by 2060, has significantly increased the demand for food, especially in agriculture (Suma, 2021). In India, this

challenge is compounded by the need to balance resource management, labor, and sustainability. Using the Internet of Things (IoT) technologies in agriculture, known as "smart agriculture," provides a promising solution to increase productivity, adapt resource use, and address the boundaries of traditional agricultural methods. The review addresses important questions such as: What are the models and methods usually used in agricultural forecasts and crop predictions? How are seasonal, regional, and environmental factors governed? What pre-processing techniques and properties are necessary to improve accuracy?

The study review mainly uses IoT-based frameworks that integrate sensors and cloud calculation to enable real-time data collection and future indicative analysis. Different models, including integrated Jordan analyzers (Shivachandran et al.), smart sensor sticks (Nayyar et al.), and wireless sensor networks (Pusatkar et al.), have been implemented to monitor and regulate agricultural parameters. Predictive frameworks rely on probabilistic methods and big data processing to forecast production, with a particular focus on monsoon patterns and crop growth stages. Preprocessing involves sensor data conditioning, threshold setting, and cloud-based storage for different types of data (e.g., soil nutrients, plant diseases).

Critical factors influencing crop yield prediction include soil nutrients (potassium, phosphorus, and nitrogen), temperature, moisture, humidity, and effective water management. Seasonal and regional variations are managed through real-time sensor monitoring and predictive analytics, which consider monsoon forecasts and crop stage analysis. For example, soil moisture sensors allow automated irrigation systems for area-specific water management (Kumar et al., 2015). Cloud computing integration improves the decision by enabling centralized data processing and expert consultation for farmers. However, challenges such as unit authentication, safety, and inequality remain. System performance is evaluated through real-time response, sensor data analysis, and system response calculations, which show improvement in irrigation control and crop monitoring with field experiments.

This underlines the importance of IoT to increase the preaching of crops. Government policy or external interventions are not deeply discussed, nor is it a comparison of the dataset. The most impressive dividend functions are average through the sensor: the soil's nutrients, moisture, temperature, and humidity. Regional and climate variations are governed by sensor-based monitoring and future frameworks that fit local conditions. It was reviewed that research has emphasized that IoT-based smart agricultural models, especially when using integrated sensors and cloud computing, are very effective at monitoring crops in India. Information about the environment updated in real time, analysis of future signs, and an automated control system are the primary factors. rights, data standards, and system-related obstacles, even though these technologies provide sufficient benefits. The aim of future research should be to standardize data formats, increase security measures, and include advanced machine learning models for greater predicted accuracy. Expanding the use of the Internet of Things in Indian agriculture can increase

flexibility, which can lead to stability, different crops, and changes in the environment. Suma, V. (2021).

- **Technological Change and Crop Yield Prediction in Indian Agriculture**

Technological progress has greatly affected the agricultural sector in India, especially in states such as Gujarat. Ashraf and Singh (2021) discover the impact of technical changes on the growth and dividend of both commercial and food-affair crops in Gujarat using time series data. It focuses on the function of literature review studies, checks agricultural forecast models, handles seasonal and regional variations, uses preprocessing techniques, identifies significant factors in crops, and compares different approaches in the Indian context. The study deals with many major research questions: What role do technical changes in the development of commercial crops make in Gujarat? How do technical changes in different crops and other entrances vary? Can you promote technology in agriculture? The hypothesis suggests that technological progress with other agricultural inputs, crops, and production patterns significantly affects.

To test this, the study employs two primary empirical models: the growth rate model and the Cobb-Douglas production function. The growth rate model examines the effect of technical changes on the crop area, production, and dividend (represented with a time trend factor) for 17 large crops using data from 1971 to 2013. The Cobb-Douglas model estimates the effects of inputs such as area sown, irrigated land, fertilizer use, labor, rural literacy, and rainfall on crop yield. Statistical tests like Ramsey RESET, VIF, and Breusch-Pagan/Cook-Weisberg tests were used during preprocessing to ensure model accuracy by checking for normality, stationarity, multicollinearity, autocorrelation, and heteroscedasticity.

The results indicate that technological change has a positive and significant impact on the yield of most crops, including cotton, sugarcane, rice, wheat, and soybeans. Important effects on the yield of the identified crop include technological progress, irrigated areas, fertilizer application, labor, rural literacy, and rainfall, although the effect varies from crop to crop. For example, the use of excessive fertilizers can reduce the fertility of the soil, and the effect of rainfall varies in crops. A prominent example discussed is the use of BT bomary in India, which reflects the positive effect of technological changes. Resistant to insects, such as BT cotton, a genetically modified variation, bollworm reduced the requirement for chemical pesticides and fertilizers, which led to low costs and high profits for farmers. This example shows how technological innovations such as BT bomary not only increase crop and profit but also contribute to more durable practices by reducing environmental effects. The study concludes that technological changes affect crop production positively in Gujarat. However, the effects of inputs such as fertilizers and labor are crop-specific, and guidelines should be done accordingly.

As the study uses traditional economic methods, future research should incorporate modern machine learning techniques to estimate crop dividends. Seasonal and regional fluctuations and

extensive government policy review require more advanced methodologies. The study emphasizes the importance of targeted technological interventions, farmer training, and sustainable agricultural practices to enhance crop diversification and sustainability in India. Singh, A. K. (2021)

- **Literature Review: Machine Learning Approaches for Crop Yield Prediction in India**

Agriculture is the largest industry in India, employs many people, and takes up 60% of the country (Gulati and Jha). Population expansion has made the exact agricultural production needed for food security and peasant support. New machine learning (ML) methods increase the accuracy and efficiency of the prediction of crop dividends. This literature undergoes 2010-2017 Indian agricultural product prediction models, methods, proclamation of methods, and important aspects. What machine learning methods are used in agricultural forecasts? How do crop models adjust to crops' seasonal and regional differences? Pretreatment and functions for agricultural data sets do the best work. How do climate and region affect agricultural product predictions? A comparison is made between ML models, modeling challenges, and evaluation norms.

Investigated studies collected data using Kaggle and data.gov.in on Indian states, crop types, seasons, and environmental variables. (Gulati & Jha). Data preparation includes extracting temperature, rainfall, humidity, and soil pH and filling missing values. The data is divided into training and testing. Linear Regression, Gradient Boosting Regressor, Random Forest Regressor, Decision Tree Regressor, Polynomial Regression, and Ridge Regression forecast crop yields.

MAE, MSE, RMSE, R-square, and cross-validation evaluate models. Gradient Boosting Regressor had the highest yield prediction cross-validation accuracy (87.9%), and Random Forest Regressor had the highest production accuracy (98.9%). Rainfall, temperature, fertilizers, pesticides, pH, and other climatic factors affect crop yield. Preprocessing missing information and extracting high/low/mean temperature, humidity, soil pH, and rainfall are essential for model effectiveness. State, district, crop year, crop area, and season are added to models to account for regional and seasonal differences. S. Veenadhari et al. estimated agricultural yield using meteorological indices, emphasizing environmental aspects.

Studies show that gradient boosting and random forest regressors can estimate Indian crop productivity. Many properties and wider pre-proclaiming models increase accuracy. Environmental variability and data quality management are difficult. The paper suggests using large datasets and advanced preprocessing to solve these problems. Future research should increase data quality, integrate environmental and policy factors, and implement advanced ML methods. Indian farmers can make informed decisions using these models and increase crop diversity and stability. Conclusions suggest that machine learning improves Indian agricultural forecasts and food security.

PART B:

Dataset Preparation

LOADING DATA:

```
[185] cropsdataset = pd.read_csv('/drive/My Drive/Colab Notebooks/cropsdataset.csv')
cropsdataset.head()
```

	Dist Code	Year	State Code	State Name	Dist Name	RICE AREA (1000 ha)	RICE PRODUCTION (1000 tons)	RICE YIELD (Kg per ha)	WHEAT AREA (1000 ha)	WHEAT PRODUCTION (1000 tons)	...	SUGARCANE PRODUCTION (1000 tons)	SUGARCANE YIELD (Kg per ha)	COTTON AREA (1000 ha)	COTTON PRODUCTION (1000 tons)	COTTON YIELD (Kg per ha)	FRUITS AREA (1000 ha)
0	1	2010	14	Chhattisgarh	Durg	777.92	1319.18	1695.77	46.12	45.87	...	1.13	256.96	NaN	NaN	NaN	2.00
1	1	2011	14	Chhattisgarh	Durg	806.55	1416.48	1756.23	44.98	50.19	...	2.06	506.02	0.08	0.02	250.00	NaN
2	1	2012	14	Chhattisgarh	Durg	807.07	1534.22	1900.97	43.87	47.69	...	1.26	252.98	0.13	0.02	153.85	1.99
3	1	2013	14	Chhattisgarh	Durg	819.44	1296.24	1581.86	41.54	43.33	...	0.93	237.45	0.05	0.01	200.00	1.90
4	1	2014	14	Chhattisgarh	Durg	834.87	1444.24	1729.89	44.25	50.96	...	3.46	602.17	0.05	0.02	400.00	NaN

5 rows x 79 columns

The dataset is filled with state names, district names, crop acreage, production, and first-row crop values. This shows the data set structure, confirming output crop data.

```
[186] cropsdataset.shape
```

↔ (2484, 79)

The output (2484, 79) indicates that the data set has 2484 rows and 79 columns, which means that there are 2484 observations (data points) and 79 functions (variables). This helps to confirm the data set size and structure.

```
cropsdataset.describe()
```

	Dist Code	Year	State Code	RICE AREA (1000 ha)	RICE PRODUCTION (1000 tons)	RICE YIELD (Kg per ha)	WHEAT AREA (1000 ha)	WHEAT PRODUCTION (1000 tons)	Unnamed: 10	KHARIF SORGHUM AREA (1000 ha)	...	SUGARCANE PRODUCTION (1000 tons)	SUGARCANE YIELD (Kg per ha)
count	2484.000000	2484.000000	2484.000000	2258.000000	2247.000000	2246.000000	2182.000000	2159.000000	946.000000	1846.000000	...	2079.000000	2066.000000
mean	270.787440	2013.496779	9.579710	149.893286	379.732314	2300.326358	139.364212	372.626082	455.541501	69.928754	...	178.713829	5889.608
std	279.103876	2.292977	4.993319	167.938403	459.018187	943.054142	185.127930	529.729178	638.655023	265.852943	...	408.592813	3145.602
min	1.000000	2010.000000	1.000000	0.010000	0.000000	90.000000	0.010000	0.010000	0.010000	0.010000	...	0.010000	0.490
25%	78.000000	2011.000000	6.000000	19.037500	33.150000	1695.455000	17.542500	24.715000	19.152500	0.800000	...	2.125000	4072.977
50%	156.000000	2013.000000	10.000000	101.860000	236.400000	2287.235000	88.625000	185.010000	157.270000	3.570000	...	18.020000	6245.540
75%	242.000000	2015.250000	12.000000	224.095000	534.540000	2815.895000	194.345000	549.445000	737.180000	19.430000	...	114.685000	7735.122
max	917.000000	2017.000000	20.000000	1154.230000	3215.010000	5653.830000	1477.850000	4305.500000	3333.330000	1853.430000	...	3252.900000	22062.300

8 rows x 77 columns

Summary statistics output provides important insights such as average, standard deviation, and range (minimum and maximum) for each numerical column. For example, the rice area (1000

hectares) means column 149.89, and the maximum value is 1154.23, reflecting the extent of data from the rice area.

```
▶ cropsdataset.isna().sum()
```

	0
Dist Code	0
Year	0
State Code	0
State Name	0
Dist Name	0
...	...
FRUITS AREA (1000 ha)	568
VEGETABLES AREA (1000 ha)	493
FRUITS AND VEGETABLES AREA (1000 ha)	556
POTATOES AREA (1000 ha)	698
ONION AREA (1000 ha)	550

79 rows x 1 columns

dtype: int64

This output shows the number of missing values for each column.

```
▶ cropsdataset.columns
```

Index(['Dist Code', 'Year', 'State Code', 'State Name', 'Dist Name', 'RICE AREA (1000 ha)', 'RICE PRODUCTION (1000 tons)', 'RICE YIELD (Kg per ha)', 'WHEAT AREA (1000 ha)', 'WHEAT PRODUCTION (1000 tons)', 'Unnamed: 10', 'KHARIF SORGHUM AREA (1000 ha)', 'KHARIF SORGHUM PRODUCTION (1000 tons)', 'KHARIF SORGHUM YIELD (Kg per ha)', 'RABI SORGHUM AREA (1000 ha)', 'RABI SORGHUM PRODUCTION (1000 tons)', 'RABI SORGHUM YIELD (Kg per ha)', 'SORGHUM AREA (1000 ha)', 'SORGHUM PRODUCTION (1000 tons)', 'SORGHUM YIELD (Kg per ha)', 'PEARL MILLET AREA (1000 ha)', 'PEARL MILLET PRODUCTION (1000 tons)', 'PEARL MILLET YIELD (Kg per ha)', 'MAIZE AREA (1000 ha)', 'MAIZE PRODUCTION (1000 tons)', 'MAIZE YIELD (Kg per ha)', 'FINGER MILLET AREA (1000 ha)', 'FINGER MILLET PRODUCTION (1000 tons)', 'FINGER MILLET YIELD (Kg per ha)', 'BARLEY AREA (1000 ha)', 'BARLEY PRODUCTION (1000 tons)', 'BARLEY YIELD (Kg per ha)', 'CHICKPEA AREA (1000 ha)', 'CHICKPEA PRODUCTION (1000 tons)', 'CHICKPEA YIELD (Kg per ha)', 'PIGEONPEA AREA (1000 ha)', 'PIGEONPEA PRODUCTION (1000 tons)', 'PIGEONPEA YIELD (Kg per ha)', 'MINOR PULSES AREA (1000 ha)', 'MINOR PULSES PRODUCTION (1000 tons)', 'MINOR PULSES YIELD (Kg per ha)', 'GROUNDNUT AREA (1000 ha)', 'GROUNDNUT PRODUCTION (1000 tons)', 'GROUNDNUT YIELD (Kg per ha)', 'SESAMUM AREA (1000 ha)', 'SESAMUM PRODUCTION (1000 tons)', 'SESAMUM YIELD (Kg per ha)', 'RAPESEED AND MUSTARD AREA (1000 ha)', 'RAPESEED AND MUSTARD PRODUCTION (1000 tons)', 'RAPESEED AND MUSTARD YIELD (Kg per ha)', 'SAFFLOWER AREA (1000 ha)', 'SAFFLOWER PRODUCTION (1000 tons)', 'SAFFLOWER YIELD (Kg per ha)', 'CASTOR AREA (1000 ha)', 'CASTOR PRODUCTION (1000 tons)', 'CASTOR YIELD (Kg per ha)', 'LINSEED AREA (1000 ha)', 'LINSEED PRODUCTION (1000 tons)', 'LINSEED YIELD (Kg per ha)', 'SUNFLOWER AREA (1000 ha)', 'SUNFLOWER PRODUCTION (1000 tons)', 'SUNFLOWER YIELD (Kg per ha)', 'SOYABEAN AREA (1000 ha)', 'SOYABEAN PRODUCTION (1000 tons)', 'SOYABEAN YIELD (Kg per ha)'], dtype=object)

The shows all 79 column names, including the state's name, district names, and various crops related to fields. It helps to confirm the names and types present in the dataset.

```

▶ cropsdataset.info()
23 MAIZE AREA (1000 ha) 2292 non-null float64
24 MAIZE PRODUCTION (1000 tons) 2307 non-null float64
25 MAIZE YIELD (Kg per ha) 2352 non-null float64
26 FINGER MILLET AREA (1000 ha) 1573 non-null float64
27 FINGER MILLET PRODUCTION (1000 tons) 1668 non-null float64
28 FINGER MILLET YIELD (Kg per ha) 1539 non-null float64
29 BARLEY AREA (1000 ha) 1779 non-null float64
30 BARLEY PRODUCTION (1000 tons) 1793 non-null float64
31 BARLEY YIELD (Kg per ha) 1819 non-null float64
32 CHICKPEA AREA (1000 ha) 2225 non-null float64
33 CHICKPEA PRODUCTION (1000 tons) 2165 non-null float64
34 CHICKPEA YIELD (Kg per ha) 2179 non-null float64
35 PIGEONPEA AREA (1000 ha) 2106 non-null float64
36 PIGEONPEA PRODUCTION (1000 tons) 2084 non-null float64
37 PIGEONPEA YIELD (Kg per ha) 2049 non-null float64
38 MINOR PULSES AREA (1000 ha) 2237 non-null float64
39 MINOR PULSES PRODUCTION (1000 tons) 2219 non-null float64
40 MINOR PULSES YIELD (Kg per ha) 2227 non-null float64
41 GROUNDNUT AREA (1000 ha) 1896 non-null float64
42 GROUNDNUT PRODUCTION (1000 tons) 1953 non-null float64
43 GROUNDNUT YIELD (Kg per ha) 1909 non-null float64
44 SESAMUM AREA (1000 ha) 2212 non-null float64
45 SESAMUM PRODUCTION (1000 tons) 2098 non-null float64
46 SESAMUM YIELD (Kg per ha) 2113 non-null float64
47 RAPESEED AND MUSTARD AREA (1000 ha) 1959 non-null float64
48 RAPESEED AND MUSTARD PRODUCTION (1000 tons) 2031 non-null float64
49 RAPESEED AND MUSTARD YIELD (Kg per ha) 2059 non-null float64
50 SAFFLOWER AREA (1000 ha) 1282 non-null float64
51 SAFFLOWER PRODUCTION (1000 tons) 1608 non-null float64
52 SAFFLOWER YIELD (Kg per ha) 1466 non-null float64
53 CASTOR AREA (1000 ha) 1833 non-null float64
54 CASTOR PRODUCTION (1000 tons) 1886 non-null float64
55 CASTOR YIELD (Kg per ha) 1916 non-null float64
56 MUSTARD AREA (1000 ha) 1757 non-null float64

```

This output suggests that the dataset has 2484 entries with 79 columns, including data types (INT64, Float64, Object) and information on non-zero calculations. This emphasizes that some columns have the missing values, such as the rice area (1000 hectares), with only 2258 non-zero entries.

cropsdataset.dtypes

	0
Dist Code	int64
Year	int64
State Code	int64
State Name	object
Dist Name	object
...	...
FRUITS AREA (1000 ha)	float64
VEGETABLES AREA (1000 ha)	float64
FRUITS AND VEGETABLES AREA (1000 ha)	float64
POTATOES AREA (1000 ha)	float64
ONION AREA (1000 ha)	float64

79 rows x 1 columns

dtype: object

The data types for each column are shown, shown that most columns are float 64 for numerical values (such as crop area and production) and objects for classified data (eg the state name and district name). This model confirms the requirement of proper prepare before training.

Exploratory Data Analysis

print(cropsdataset.describe()) # Summary statistics for numerical data

	Dist Code	Year	State Code	RICE AREA (1000 ha) \
count	2484.000000	2484.000000	2484.000000	2258.000000
mean	278.787440	2013.495779	9.579710	149.893206
std	279.103876	2.292977	4.993319	167.938403
min	1.000000	2010.000000	1.000000	0.010000
25%	78.000000	2011.000000	6.000000	19.037500
50%	156.000000	2013.000000	10.000000	101.860000
75%	242.000000	2015.250000	12.000000	224.095000
max	917.000000	2017.000000	20.000000	1154.230000

	RICE PRODUCTION (1000 tons)	RICE YIELD (Kg per ha) \
count	2247.000000	2246.000000
mean	379.732314	2300.326358
std	459.018187	943.054142
min	0.000000	90.000000
25%	33.150000	1695.455000
50%	236.400000	2287.235000
75%	534.540000	2815.895000
max	3215.010000	5653.830000

	WHEAT AREA (1000 ha)	WHEAT PRODUCTION (1000 tons)	Unnamed: 10 \
count	2182.000000	2159.000000	946.000000
mean	139.364212	372.626082	455.541501
std	185.127930	529.729178	638.655023
min	0.010000	0.010000	0.010000
25%	17.542500	24.715000	19.152500
50%	88.625000	185.010000	157.270000
75%	194.345000	549.445000	737.180000

A crop area, production data and return are included in this data set in many states in India. The calculation provides information on the number of elements in each column that is not empty. Some elements, such as rice area (1000 hectares) and rice production (1000 tonnes), are missing data. These areas include rice areas. If you want to know what the average number in each column is, you can use the meaning. For example, specific rice dividends, measured in per hectare kilos, are approximately 2,300.33 kg per hectare. Standard deviation, or STD, is a volume that indicates

how the values are spread in relation to the value. Some properties, such as rice sector and rice production, undergo more frequent changes than others. At the same time when the minimum and maximum value reflects the data boundary, they also clearly make it clear that there are important variations between places. When it comes to this scenario, the rice area can occur from 0.01% to 1154.23 hectares. With the use of these summary data, you will be able to identify patterns, potential impacts and crop production and changes in crop production and dividends. To train the model and prepare further data, you can use this data.

Outliers

✓ **** box plots for detecting outliers ****

```
def plot_outliers(data):
    """
    Plots box plots for each numerical column to visualize outliers.
    """
    numerical_columns = data.select_dtypes(include=['int64', 'float64']).columns
    num_columns = len(numerical_columns)

    cols = 3
    rows = math.ceil(num_columns / cols)

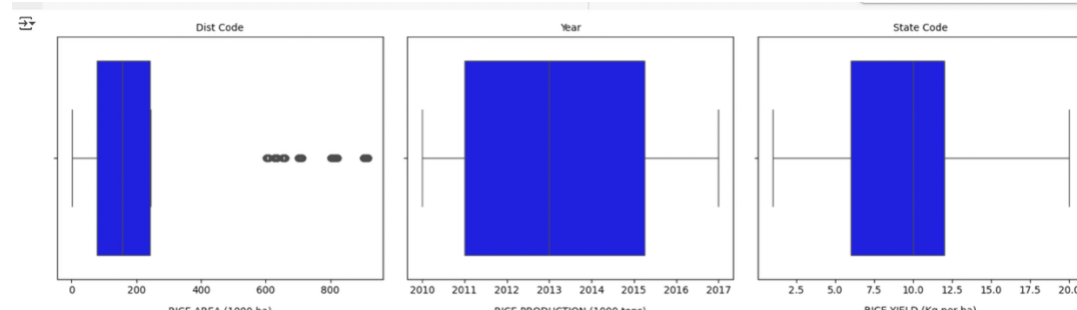
    plt.figure(figsize=(cols * 5, rows * 4))

    for i, column in enumerate(numerical_columns, 1):
        plt.subplot(rows, cols, i)
        sns.boxplot(x=cropsdataset[column], color='blue')
        plt.title(f'{column}', fontsize=10)
        plt.xlabel('')

    plt.tight_layout()
    plt.show()

# Call the function
plot_outliers(cropsdataset)
```

Box Plot Overview: Each plot shows a number other than your information. The interquartile range (IQR) is shown with the box, with a line inside the box indicating the median. The "mustache" of the plot is not in, usually used to show the clearing.



Outliers: My plot has many features that are outside the mustache, which is a sign of an outlier. These are figures that are not in the general category of data and may require them to move on or cope with preaching.

scatter plots

scatter plots display

```
import seaborn as sns
import matplotlib.pyplot as plt

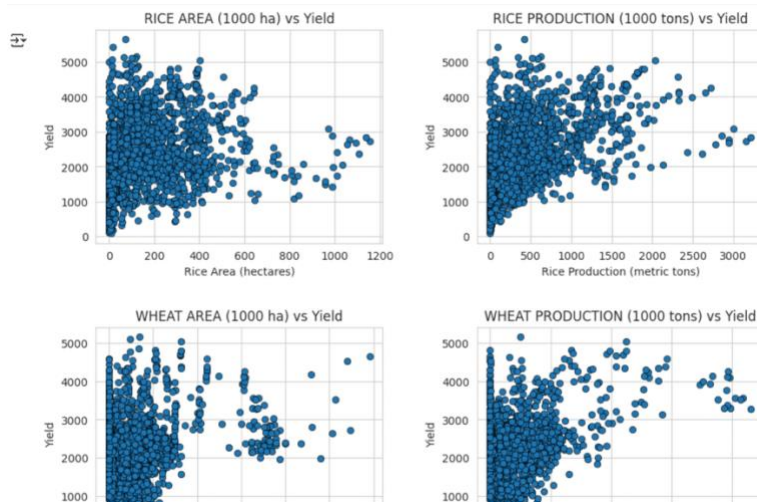
# Set style for the plots
sns.set_style("whitegrid")

# Select relevant columns based on your dataset
features_with_units = {
    'RICE AREA (1000 ha)': 'Rice Area (hectares)',
    'RICE PRODUCTION (1000 tons)': 'Rice Production (metric tons)',
    'WHEAT AREA (1000 ha)': 'Wheat Area (hectares)',
    'WHEAT PRODUCTION (1000 tons)': 'Wheat Production (metric tons)'
}

# Number of plots and layout setup
num_plots = len(features_with_units)
fig, axes = plt.subplots((num_plots + 1) // 2, 2, figsize=(10, 8))
axes = axes.flatten()

# Plot the scatterplots
for i, (feature, label) in enumerate(features_with_units.items()):
    sns.scatterplot(x=cropsdataset[feature], y=cropsdataset['RICE YIELD (Kg per ha)'],
                   axes[i].set_title(f'{feature} vs Yield'))
    axes[i].set_xlabel(label)
    axes[i].set_ylabel('Yield')

plt.tight_layout(pad=3.0)
```



As you can see, scatter plots show how crop areas, crop production, and rice production are related. For rice area, rice production, wheat area, and wheat production, each plot shows a number next to rice production (kg per hectare).

Rice area versus returns plots show that large rice areas do not always mean high dividends. This suggests that disability or regional factors can affect the yield regardless of the size of the area. Rice production versus return plots shows a uniform pattern, with no clear tendency between production and yield. This suggests that only other factors other than production volume can affect the dividend.

The plots for wheat area vs. yield and wheat production vs. yield show that both production and area have different effects on wheat yield.

The visualizations:

crop distribution in India across different states, years (seasons), and crop types based on the area they occupy.

```
#visualize the distribution of crops across states and years (seasons) as well as the di:

# Custom color palette for consistency
palette = sns.color_palette("Blues", n_colors=5)

# Create subplots for state, season, and crop type distribution
fig, axes = plt.subplots(2, 2, figsize=(16, 12), gridspec_kw={'height_ratios': [1, 0.5]})
axes = axes.flatten()

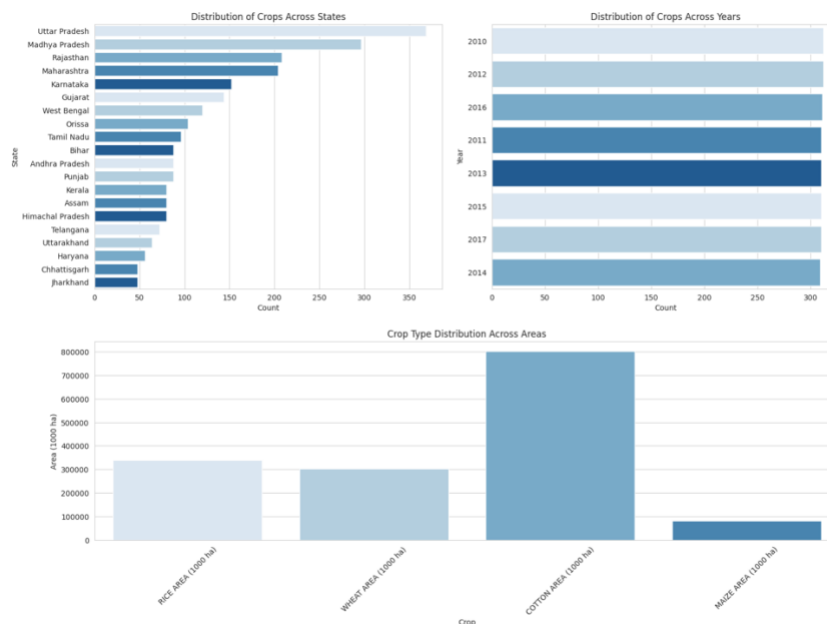
# Plot state distribution
sns.countplot(y=cropsdataset['State Name'], order=cropsdataset['State Name'].value_counts())
axes[0].set_title('Distribution of Crops Across States')
axes[0].set_xlabel('Count')
axes[0].set_ylabel('State')

# Plot season distribution
sns.countplot(y=cropsdataset['Year'], order=cropsdataset['Year'].value_counts().index, ax:
axes[1].set_title('Distribution of Crops Across Years')
axes[1].set_xlabel('Count')
axes[1].set_ylabel('Year')

# Removing unnecessary subplots
axes[2].remove()
axes[3].remove()

# Plot crop type distribution
crop_counts = cropsdataset[['RICE AREA (1000 ha)', 'WHEAT AREA (1000 ha)', 'COTTON AREA
crop_counts = crop_counts.reset_index()
```

OUTPUT:



Key Insights from the Plots:

1. How crops are spread: The first plot is a counting plot showing how crops are scattered in Indian states. For a long time, most of the crop yield in Uttar Pradesh, Madhya Pradesh, Rajasthan, and Maharashtra. This means that these states make a big difference in the production of India's cultivation.
2. How crops have spread over the years (the weather): Another plot shows how food production has changed from 2010 to 2017; most crops were cultivated in 2013. This can be a year with a special program in many crops or agriculture showing how production changes with weather.
3. Spread of crop types in different regions: The third plot is the one that shows what kind of country is spread with the amount of land. The crop area consists mostly of rice, cotton, and wheat. Cotton takes up as much space as possible, which shows how important it is for India's agricultural industry.

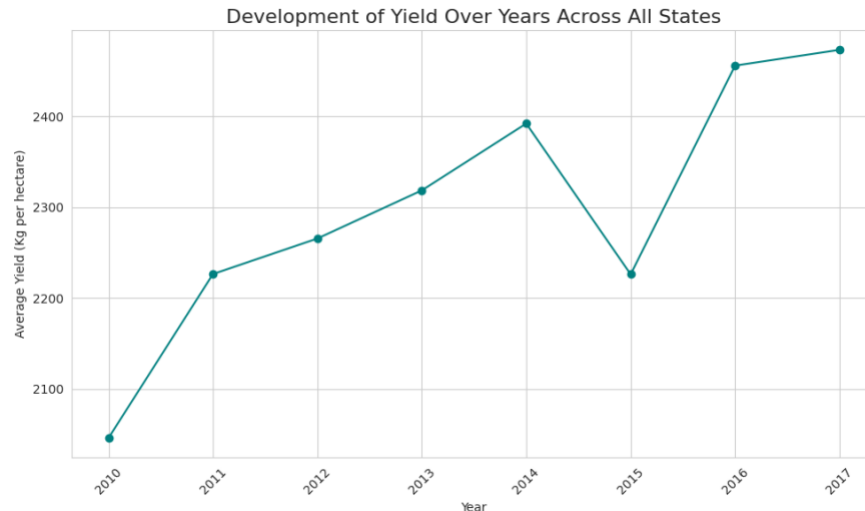
These plots tell us a lot about how crops are spread over conditions, years, and types. Depending on what they say, the most important crops in India are rice, wheat, and cotton, with Uttar Pradesh and Madhya Pradesh getting the most out of these. These facts can help you find out how agriculture works in different parts of the country and how the weather and different states affect food yields.

The line plot you generated visualizes the average rice yield (kg per hectare) over the plot and highlights the following trends:

```
#Yield Development Over the Years Across All States.
#visualize how the average yield has changed over the years across all states in your dataset

# Grouping by 'Year' and calculating the mean yield
yearly_yield = cropsdataset.groupby('Year')['RICE YIELD (Kg per ha)'].mean().reset_index()

# Plotting the yield trend over the years
plt.figure(figsize=(10, 6))
plt.plot(yearly_yield['Year'], yearly_yield['RICE YIELD (Kg per ha)'], marker='o', linestyle='solid')
plt.title('Development of Yield Over Years Across All States', fontsize=16)
plt.xlabel('Year')
plt.ylabel('Average Yield (Kg per hectare)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Important points:

1. From 2010 to 2013, the return gradually increased, which showed that the methods, weather, or policy changes in farming were better, which led to increased production of rice.
2. In 2014, the return fell rapidly, which may mean that something poorly affected production, such as drought, a bad monsoon, or other problems in the region.
3. 2015–2017: The dividend is increasing, and when it reaches the highest point in 2017. This may mean that things will be better after the decline in 2014, or it could mean agricultural equipment, irrigation, or fertilizer use has improved.

Finally, this plot makes it easy to see how the rice yield has changed over time, which helps you detect trends and outlier. More research can be done to know why the return decreased in 2014 and why it has grown since 2015.

scaled yield of rice over the years across all states.

```

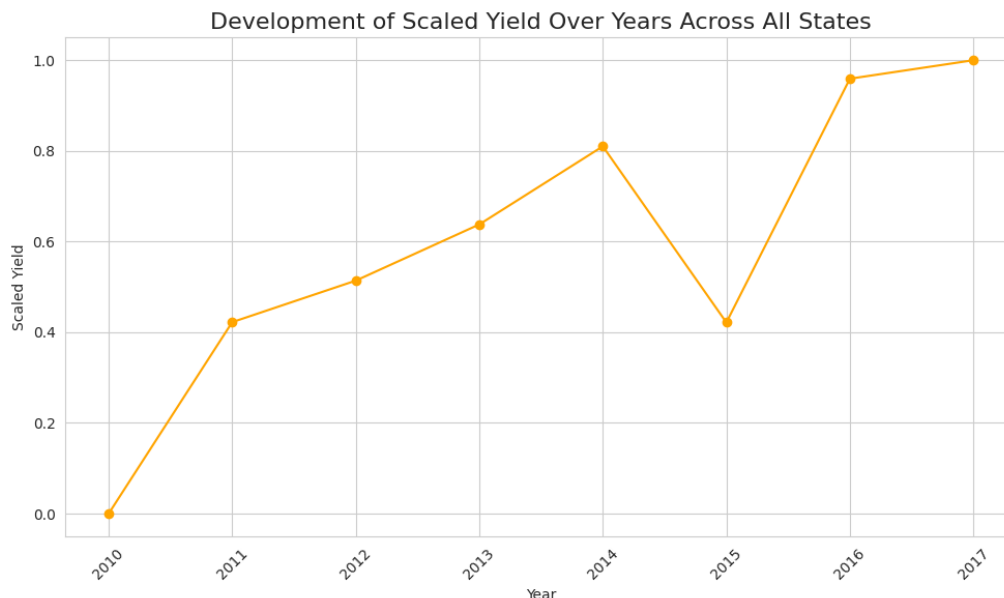
#Development of Scaled Yield Over the Years Across All States
#the average yield is scaled (typically by normalizing the data) before plotting

# Grouping by 'Year' and calculating the mean yield
yearly_yield = cropsdataset.groupby('Year')['RICE YIELD (Kg per ha)'].mean().re:

# Scaling the yield values using MinMaxScaler
scaler = MinMaxScaler()
yearly_yield['Yield_scaled'] = scaler.fit_transform(yearly_yield[['RICE YIELD (I

# Plotting the scaled yield trend over the years
plt.figure(figsize=(10, 6))
plt.plot(yearly_yield['Year'], yearly_yield['Yield_scaled'], marker='o', linestyle='solid')
plt.title('Development of Scaled Yield Over Years Across All States', fontsize=12)
plt.xlabel('Year')
plt.ylabel('Scaled Yield')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



1. Dividend Normalization: MinMaxscaler produces return values from 0 to 1.
2. Trend analysis: From 2010 to 2011, scaled return shows a significant increase, which potentially indicates an increase in agricultural function or climatic conditions.
The reduction in 2014 depicts a comparable pattern seen in the raw return conspiracy, which may reflect side effects such as meteorological events or agricultural diseases.
Rapid growth in 2017 may indicate the use of more efficient agricultural practices.

Conclusion: Normalization of data facilitates analysis of ups and downs in dividends over time and emphasizes relative changes instead of full values. It provides a more attractive perspective on many years of trends.

Development of Yield Over the Years by State

These plots provide valuable insight into annual return changes, which helps identify specific trends and potential causes of changes. It also shows regional differences in agricultural productivity. This analysis may guide additional studies or intervention in state-specific agricultural practices

```
#Development of Yield Over the Years by State
#This code plots the development of yield over the years for multiple sta
#It helps compare how each state's yield has evolved over the years, which

# Get unique states in the dataset
states = cropsdataset['State Name'].unique()

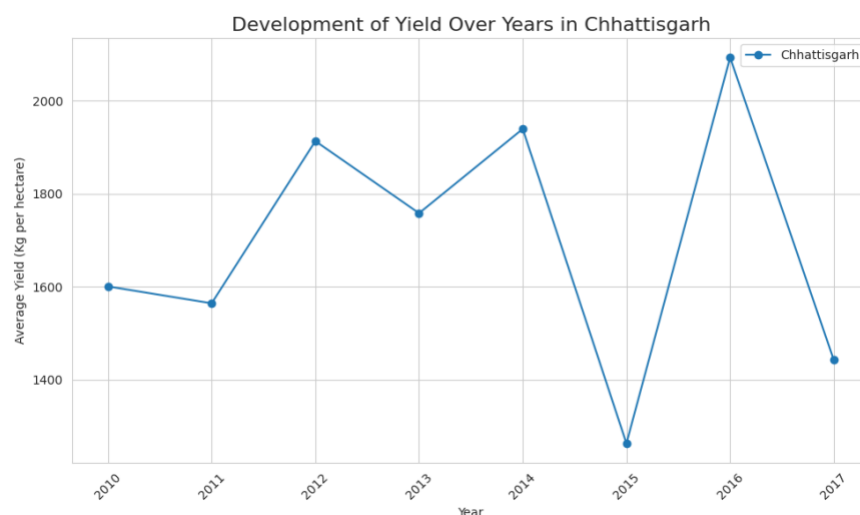
# Loop through each state and plot the yield trend over the years
i = 0
for state in states:
    state_data = cropsdataset[cropsdataset['State Name'] == state]

    # Group by year and calculate the mean yield for the state
    yearly_yield = state_data.groupby('Year')['RICE YIELD (Kg per ha)'].m

    i += 1

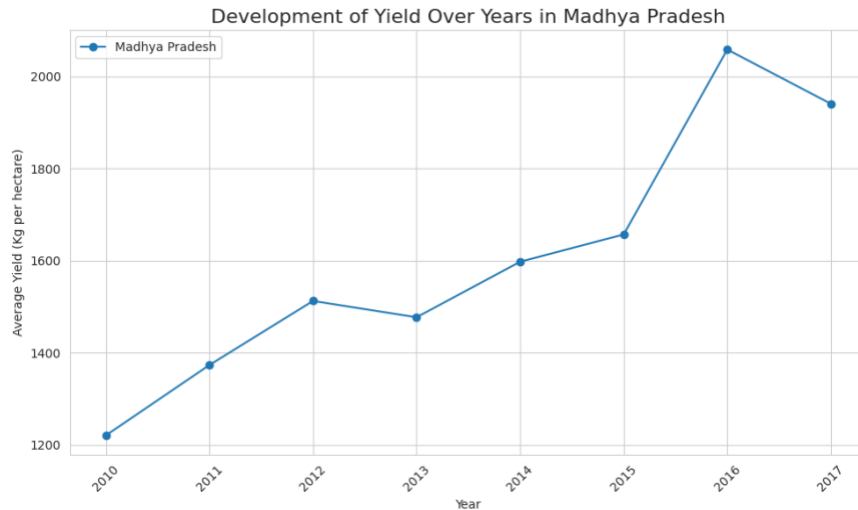
    # Plotting the yield trend for the first 3 states
    if i > 3:
        break

    plt.figure(figsize=(10, 6))
    plt.plot(yearly_yield['Year'], yearly_yield['RICE YIELD (Kg per ha)'])
    plt.title(f'Development of Yield Over Years in {state}', fontsize=16)
    plt.xlabel('Year')
    plt.ylabel('Average Yield (Kg per hectare)')
    plt.grid(True)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.legend()
    plt.show()
```

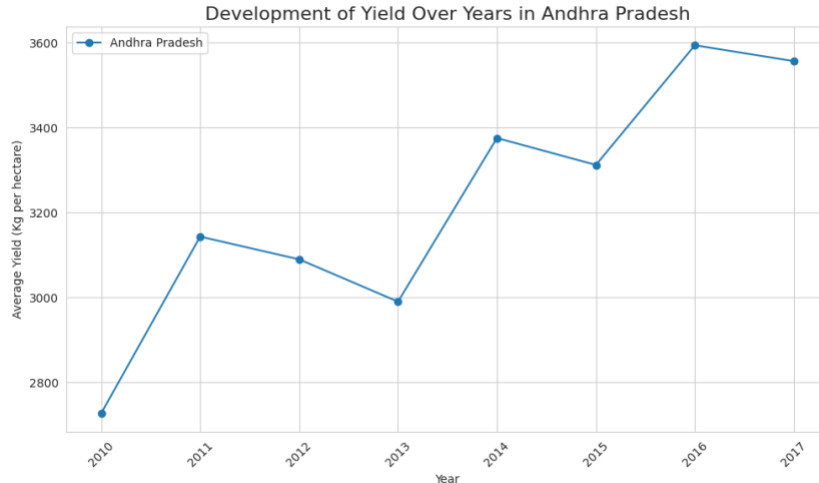


Chhattisgarh:

- The return from 2010 to 2017 with a rapid increase in 2016, there are many ups and downs, followed by a decline in 2017. This trend raises the possibility of regional repercussions, such as shifts in farming methods or altered weather patterns impacting crop yields.



Madhya Pradesh: • The return has been going up since 2010, with a big jump in 2016. This means that better ways of farming or taking care of crops could lead to more crops in the end.



Andhra Pradesh: In 2015, there was a huge return, and in 2016, things became a lot more stable. This could show how the climate or the economy affected these years.

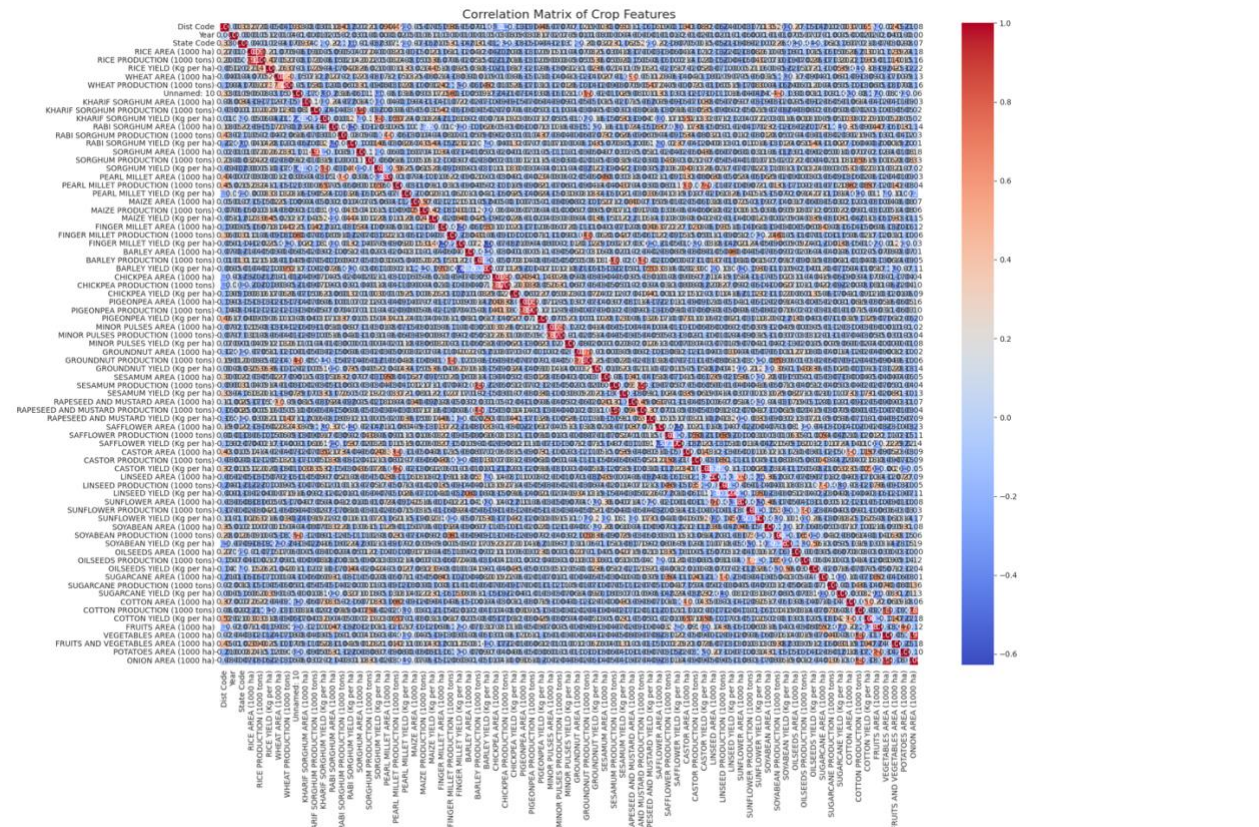
heatmap of the correlation matrix

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Select only numeric columns from the dataset (this avoids non-numeric column errors)
numeric_df = cropsdataset.select_dtypes(include=[np.number])

# Calculate the correlation matrix for the numeric columns
correlation_matrix = numeric_df.corr()

|
Plotting the heatmap of the correlation matrix
plt.figure(figsize=(20, 16)) # Increase the figure size to make it bigger
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix of Crop Features', fontsize=16)
plt.tight_layout()
plt.show()
```



One thing that stands out is a significantly positive correlation between different crop parameters. For example, it is not surprising to see that the area under agriculture for rice has a direct impact on the dividend.

- There is a strong correlation between the production of wheat and the wheat sector, which suggests that both land distribution and wheat production follow comparable trends.
- Reverse correlations: This means when the value of a function increases, the value of another function is reduced. It is known as a negative correlation. For example, in a place where different types of crops compete for the same land resources, it is possible to look at negative correlations between the fruit area and the wheat area.
- Groups of properties that are strongly correlated: As an example, oil seeds, cotton, and vegetables are all examples of crop types that cling simultaneously, suggesting that they show continuous patterns in the state from year to year or state to state.

Distribution Plots (with KDE)

```

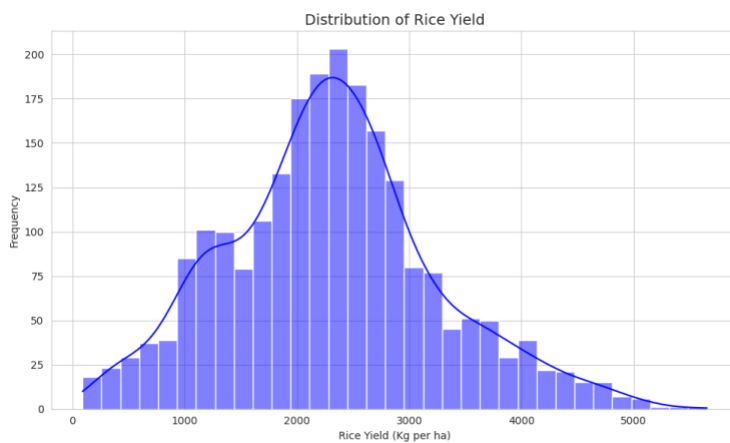
#help you see how a feature like crop yield or area is distributed. Are the values clustered?
#Is the distribution skewed?
#This can give insights into the nature of the data and help guide any necessary transformations or fea

# Distribution of Rice Yield
plt.figure(figsize=(10, 6))
sns.histplot(cropsdataset['RICE YIELD (Kg per ha)'], kde=True, color='blue')
plt.title('Distribution of Rice Yield', fontsize=14)
plt.xlabel('Rice Yield (Kg per ha)')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

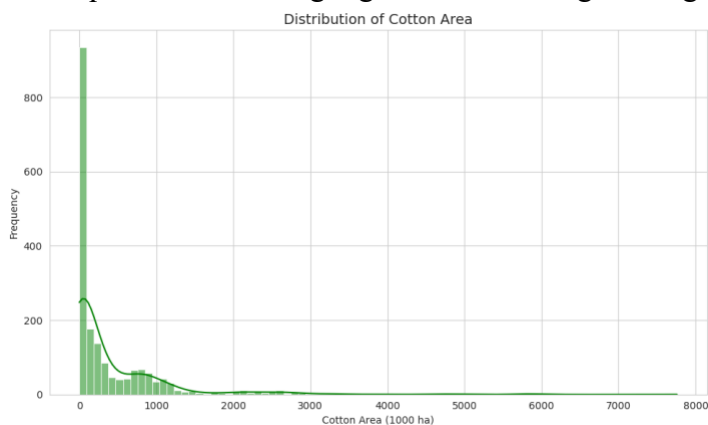
# Distribution of Cotton Area
plt.figure(figsize=(10, 6))
sns.histplot(cropsdataset['COTTON AREA (1000 ha)'], kde=True, color='green')
plt.title('Distribution of Cotton Area', fontsize=14)
plt.xlabel('Cotton Area (1000 ha)')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# Distribution of Maize Production
plt.figure(figsize=(10, 6))
sns.histplot(cropsdataset['MAIZE PRODUCTION (1000 tons)'], kde=True, color='orange')
plt.title('Distribution of Maize Production', fontsize=14)
plt.show()

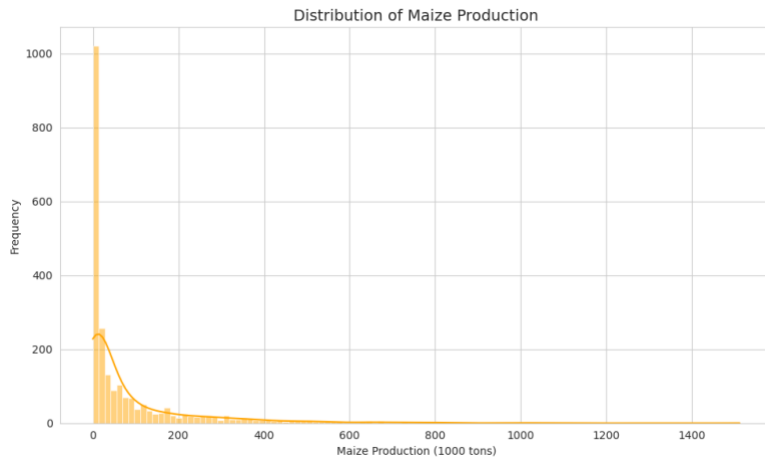
```



Rice Yield: It follows a distribution with a peak around a certain value and possibly indicates a cluster area with uniform dividends. The size of the distribution can help identify whether the data is oblique, and such insight guides feature engineering.



Cotton Area: A very sloping distribution suggests that most cotton cultivation occurs in small areas, with some areas being very large areas under agriculture. This delivery model can affect behavior and require logarithmic changes for better modeling performance.



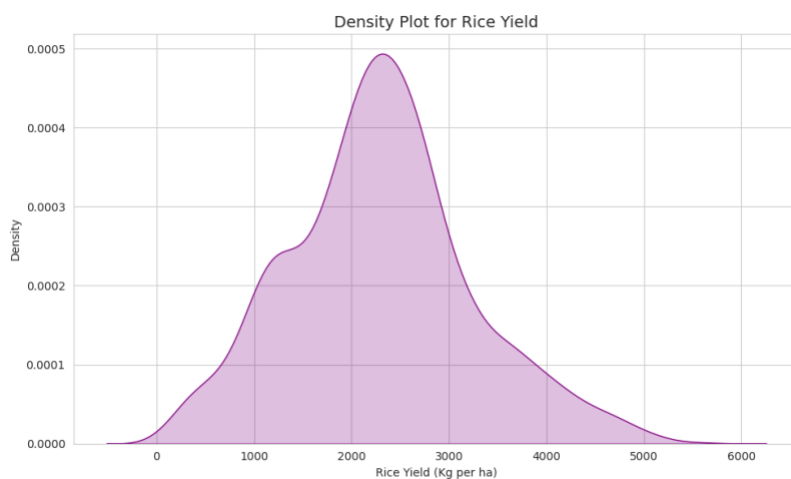
Maize Production: Like the cotton area, corn production is likely to be correct; the cluster has been clustered in the lower area with most values, but some representing quite large presentations with outliers.

Visualizing the Distribution Using Density Plots (KDE)

```
#Visualizing the Distribution Using Density Plots (KDE)
#A Density Plot (or Kernel Density Estimate plot) is used to visualize the distribution of a continuous variable.

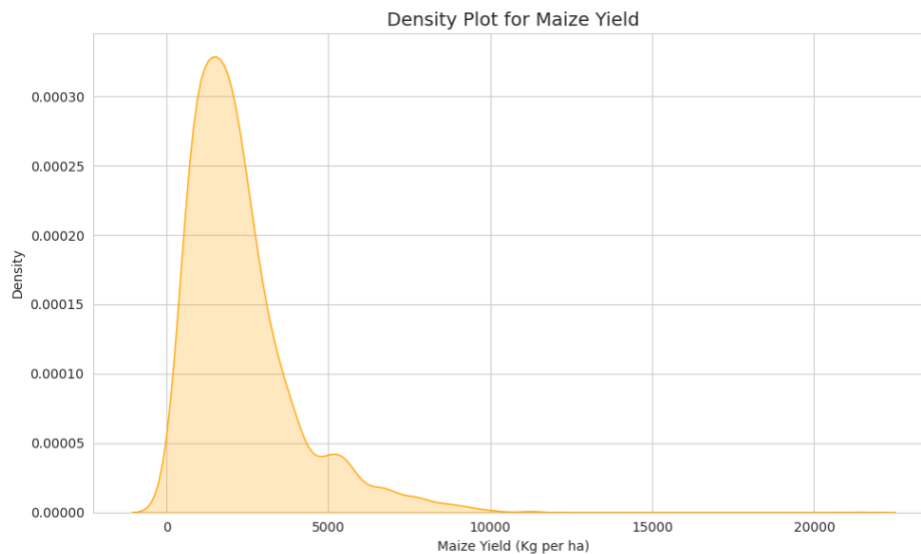
# KDE for Rice Yield
plt.figure(figsize=(10, 6))
sns.kdeplot(cropsdataset['RICE YIELD (Kg per ha)'], fill=True, color='purple') # Changed 'show' to 'fill'
plt.title('Density Plot for Rice Yield', fontsize=14)
plt.xlabel('Rice Yield (Kg per ha)')
plt.ylabel('Density')
plt.tight_layout()
plt.show()

# KDE for Maize Yield
plt.figure(figsize=(10, 6))
sns.kdeplot(cropsdataset['MAIZE YIELD (Kg per ha)'], fill=True, color='orange') # Changed 'show' to 'fill'
plt.title('Density Plot for Maize Yield', fontsize=14)
plt.xlabel('Maize Yield (Kg per ha)')
plt.ylabel('Density')
plt.tight_layout()
plt.show()
```



The distribution of rice yield in the dataset is imagined using a histogram with a core density estimate (KDE) plot. The graph shows that the rice yield is mainly concentrated around 2000 to

3000 kg per hectare, indicating that most rice dividends are in this area. Distribution is the right elasticity, which means that there are fewer data points with much higher yield than more common low dividends. This oblique may indicate the presence of extreme outliers or regions where the rice yield is exceptionally high. The KDE plot, which provides a smooth distribution curve, confirms the specific return area and exposes the trend of return on the cluster around the average.



In the corn production distribution, we notice a right -wing pattern, with most corn production values that are split at the lower lower ends (about 0 to 2000 tonnes). This suggests that most of the comments provide relatively low corn. However, there is a long tail on the right, which reflects some areas where corn production reaches very high values. The KDE plot confirms this observation by showing a peak almost low values and by turning to high values, which further states that even though production in most areas is low, there are some areas with high dividends. Spellism can be an indication of outlars or areas with very productive corn cultivation.

Data Preprocessing

Feature Engineering

feature engineering Function technique is done by creating new features based on existing data. For example, the yield per hectare of rice per hectare is calculated by dividing the production of rice with rice area, yielding from a generalized productivity value. Log transformation is used on sloping functions such as rice and cotton production to reduce the effect of extreme values and distribute data normally. Finally, irrelevant columns are dropped and the cracked variables occur for machine learning compatibility

Feature Engineering

```
import numpy as np

# 1. Create new features
cropsdataset['RICE YIELD PER HECTARE'] = cropsdataset['RICE PRODUCTION (1000 tons)'] / cropsdataset['RICE AREA (1000 ha)']
cropsdataset['WHEAT YIELD PER HECTARE'] = cropsdataset['WHEAT PRODUCTION (1000 tons)'] / cropsdataset['WHEAT AREA (1000 ha)']
cropsdataset['COTTON YIELD PER HECTARE'] = cropsdataset['COTTON PRODUCTION (1000 tons)'] / cropsdataset['COTTON AREA (1000 ha)']
cropsdataset['MAIZE YIELD PER HECTARE'] = cropsdataset['MAIZE PRODUCTION (1000 tons)'] / cropsdataset['MAIZE AREA (1000 ha)']

# 2. Log transformation of highly skewed features
cropsdataset['log_RICE_PRODUCTION'] = np.log(cropsdataset['RICE PRODUCTION (1000 tons)'] + 1)
cropsdataset['log_COTTON_PRODUCTION'] = np.log(cropsdataset['COTTON PRODUCTION (1000 tons)'] + 1)

# 3. Drop irrelevant columns
cropsdataset.drop(columns=['State Code', 'Dist Code', 'Unnamed: 10'], inplace=True)

# 4. Encode categorical features
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
cropsdataset['State Name'] = label_encoder.fit_transform(cropsdataset['State Name'])

# 5. Handling Yearly data
cropsdataset['Years_Since_First'] = cropsdataset['Year'] - cropsdataset['Year'].min()

# Check the first few rows to verify the changes
print(cropsdataset.head())
```

OUTPUT

```
(5)
0      0.32 ...
1      0.20 ...
2      0.49 ...
3      4.15 ...
4      4.15 ...

FRUITS AND VEGETABLES AREA (1000 ha)  POTATOES AREA (1000 ha) \
0      34.300      0.11
1      14.155      0.12
2      35.690      0.12
3      34.740      0.11
4      14.155      0.33

ONION AREA (1000 ha)  RICE YIELD PER HECTARE  WHEAT YIELD PER HECTARE \
0      0.49      1.695778      0.994579
1      0.54      1.756221      1.115829
2      0.54      1.900975      1.087075
3      0.50      1.581861      1.043091
4      0.51      1.729898      1.151638

COTTON YIELD PER HECTARE  MAIZE YIELD PER HECTARE  log_RICE_PRODUCTION \
0      0.159543      1.460568      7.185523
1      0.250000      1.286501      7.256636
2      0.153846      1.538462      7.336429
3      0.200000      1.741758      7.167994
4      0.400000      1.452736      7.276031

log_COTTON_PRODUCTION  Years_Since_First
0      2.876949      0
1      0.019803      1
2      0.019803      2
3      0.009950      3
4      0.019803      4

[5 rows x 83 columns]
```

The output data shows the results of data preprocessing and feature engineering. After filling the missing values, the dataset for remaining zero values is tested, confirming that no missing data is left. The first pairs of dataset are displayed, which reveals the recently added features. These include calculating log-transform functions such as rice yield per hectare rice, by dividing the production of rice according to rice area and for cleaned data. Year_since_first columns provide a measurement of time differences from the first year of the dataset. The output also confirms the release of irrelevant columns such as condition code, district code and anonymous: 10, which was unnecessary for analysis. The data now has only relevant functions, with a cleaner and more structured shape, it is ready for modeling and analysis. Changes help improve the general quality of machine learning functions.

Scaling and Normalization

In this code, scaling or general normalization is performed on numerical columns with datasets. This process begins to use `select_dtypes` (`include=[np.number]`). The standards for Scikit-Learn are then initiated to standardize data. The standardization replaces the data with a standard deviation of 0 and 1. This is especially useful when the dataset has very different scales or devices in different properties, which can cause problems during model training. The scale is used on numerical columns using the `scale`. `Fit transform()`, which fits the data model and then uses change. Finally, the mean and standard deviations for the converted data are printed, which confirms that all numerical columns have been properly standardized.

```
from sklearn.preprocessing import StandardScaler

# Select only numerical columns for scaling
numerical_cols = cropsdataset.select_dtypes(include=[np.number]).columns

# Remove non-numerical columns that should not be standardized
numerical_cols = [col for col in numerical_cols if col not in ['State Name']]

# Initialize the StandardScaler
scaler = StandardScaler()

# Apply standardization to the numerical columns
cropsdataset[numerical_cols] = scaler.fit_transform(cropsdataset[numerical_cols])

# Verify the mean and std after standardization
print(cropsdataset[numerical_cols].mean()) # Should be close to 0
print(cropsdataset[numerical_cols].std()) # Should be close to 1 for StandardScaler
```

Result:

```
Year          5.488900e-15
RICE AREA (1000 ha)  1.258610e-16
RICE PRODUCTION (1000 tons)  4.576765e-17
RICE YIELD (Kg per ha) -4.004669e-17
WHEAT AREA (1000 ha)  4.576765e-17
...
COTTON YIELD PER HECTARE  8.581434e-18
MAIZE YIELD PER HECTARE  0.000000e+00
log_RICE_PRODUCTION      5.492118e-16
log_COTTON_PRODUCTION    6.865147e-17
Years_Since_First       -3.647109e-17
Length: 81, dtype: float64
Year          1.000201
RICE AREA (1000 ha)  1.000201
RICE PRODUCTION (1000 tons)  1.000201
RICE YIELD (Kg per ha)  1.000201
WHEAT AREA (1000 ha)  1.000201
...
COTTON YIELD PER HECTARE  1.000201
MAIZE YIELD PER HECTARE  1.000201
log_RICE_PRODUCTION      1.000201
log_COTTON_PRODUCTION    1.000201
Years_Since_First       1.000201
Length: 81, dtype: float64
```

The output reflects the interpretation and the standard deviations between numerical columns after implementation of standardization. As expected, the mean values are very close to 0 (eg 5.48E-15, which is 0 due to liquid point accuracy). Similarly, the standard deviation for all numerical

columns is approximately 1 (eg 1,000201), which confirms that the standards have effectively standardized the data. This change ensures that data is focused around 0 and has the same variance in all numerical properties, making it suitable for most machine learning algorithms that are sensitive to scaling systems.

Scaling with min-max

The min-max scaling method changes the data by making the functions larger or less so that all values are in a certain area, usually between 0 and 1. This is especially useful when data has functions in different sizes, as it ensures that no features stand out because it is larger. This change is made by Scikit-Learn with MinMaxScaler.

1. Establishment of MinMaxScaler: MinMaxScaler is set to scale each column with numbers, which is done by removing the lowest value and dividing the area (maximum minute). Enter min-max scaling: It sets scaling for numerical fields in the dataset. Fit_transform() year, rice area (1000 hectares), rice production (1000 tonnes), and other columns as the columns on a 0-1 scale. 3. Verification of output: The scales of the first lines of the dataset have been replaced. These figures are now between 0 and 1, and the original numbers are adjusted. This step ensures that all functions have uniform scales, which helps with machine learning methods that care about the size of functions.

```
#Min-Max Scaling:
from sklearn.preprocessing import MinMaxScaler

# Initialize the MinMaxScaler
min_max_scaler = MinMaxScaler()

# Apply min-max scaling to the selected columns
cropsdataset[numerical_cols] = min_max_scaler.fit_transform(cropsdataset[numerical_cols])

# Check the first few rows after scaling
print(cropsdataset.head())
```

	Year	State Name	Dist Name	RICE AREA (1000 ha)	\
0	0.000000		3 Durg	0.673970	
1	0.142857		3 Durg	0.698775	
2	0.285714		3 Durg	0.699225	
3	0.428571		3 Durg	0.709943	
4	0.571429		3 Durg	0.723311	

	RICE PRODUCTION (1000 tons)	RICE YIELD (Kg per ha)	WHEAT AREA (1000 ha)	\
0	0.410319	0.288609	0.031201	
1	0.440583	0.299475	0.030430	
2	0.477205	0.325490	0.029678	
3	0.403184	0.268135	0.028102	
4	0.449218	0.294741	0.029936	

	WHEAT PRODUCTION (1000 tons)	KHARIF SORGHUM AREA (1000 ha)	\
0	0.010652	0.000119	
1	0.011655	0.000135	
2	0.011074	0.000372	
3	0.010062	0.001921	
4	0.011834	0.001921	

The output shows the first rows of datasets after applying min-MAX scaling. The values of each numerical column are expanded to fall between 0 and 1, as expected by changes. For example: 1 year: The column year, which originally had values from 2010 to 2017, now has values between 0 and 1. The scaling is done by reducing the minimum year (2010) and dividing with the total limit (2017-2010), and thus a normal scale mapping.

Rice yield and production: The rice yield (kg per hectare) and rice production (1000 tons) columns also show scaled numbers. The dividend in the first row, which used to be 1695.77 kg per hectare, is now 0.288609 on a scale of 0 to 1.

All other numerical functions in the dataset, like wheat fields and corn output, are scaled here. No function should dominate model training, and this scale ensures that.

Splitting the dataset into training and testing sets

Step to share the dataset:

1. Define features (x) and dimensions (y):

- Features (X) are input variables that you want to use to predict the target variable.
- The target (y) is the variable you try to predict (e.g., rice yield (kilo per hectare)).

2. Share the data:

To split your data, use `train_test_split` from `sklearn.model_selection`.

Explanation:

- You have 410 total columns in your dataset (including the target column rice yield (including kg per hectare)), and after leaving the target column, the number of functions (columns) is 409.
- You share the data set by 80% for training and 20% for testing.
 - o Training data (X_TRAIN): 80% of your data will be used for training. So 80% of the 2484 total rows give you samples from 1987 in X_TRAIN (functions).
 - o Test data (x_test): The remaining 20% data gives you 497 samples in x_test (functions).

Such:

- X_TRAIN will have 1987 rows and 409 columns.
- X_TEST will have 497 rows and 409 columns.

This is what you will expect based on the 80-20 section.

Model implementation

Handling NaN and Infinite Values in Data

```

▶ # Check for NaN or Infinite values in X_train and X_test
print(np.isnan(X_train_scaled).sum()) # Check for NaNs
print(np.isinf(X_train_scaled).sum()) # Check for infinities

# If there are any, remove or replace them
X_train_scaled = np.nan_to_num(X_train_scaled) # Replace NaNs with 0
X_test_scaled = np.nan_to_num(X_test_scaled)

```

```

↗ 35516
  0

```

The code checks for NaN and infinite values in the scaled training (`X_train_scaled`) and testing (`X_test_scaled`) datasets using `np.isnan()` and `np.isinf()`. It counts how many NaN or infinite values exist in the datasets. If found, `np.nan_to_num()` replaces NaN or infinite values with 0. The output shows that there are 35,516 NaN values in the training set and no infinite values, indicating the need for imputation. After replacement, the datasets are ready for use in machine learning models, ensuring no invalid values interfere with model training.

checks using `np.isnan()` and `np.isinf()` in dataset scaled training for NAN and endless values. It requires how many nan or infinite values are present in the dataset. If the discovery replaces `np.nan_to_num()` nan or infinite values from 0. The output shows that the training kit has 35,516 nan values and there is no infinite value, reflecting the need for copying. After compensation, data set machines are ready for use in learning models and ensure that no invalid value models interfere with training.

```

▶ # Check for NaN values in the original dataset (before scaling)
print(X_train.isnull().sum()) # Check for NaNs in the training data
print(X_test.isnull().sum()) # Check for NaNs in the test data

```

```

↗ Year      0
  State Name 0
  Dist Name  0
  RICE AREA (1000 ha) 174
  RICE PRODUCTION (1000 tons) 179
  ...
  COTTON YIELD PER HECTARE 694
  MAIZE YIELD PER HECTARE 164
  log_RICE_PRODUCTION 179
  log_COTTON_PRODUCTION 658
  Years_Since_First 0
  Length: 82, dtype: int64
  Year      0
  State Name 0
  Dist Name  0
  RICE AREA (1000 ha) 52
  RICE PRODUCTION (1000 tons) 58
  ...
  COTTON YIELD PER HECTARE 192
  MAIZE YIELD PER HECTARE 42
  log_RICE_PRODUCTION 58
  log_COTTON_PRODUCTION 182
  Years_Since_First 0
  Length: 82, dtype: int64

```

Each feature shows the number of missing values in each function for both output and test data. For example, the training set has 174 lack of values in the rice area (1000 hectares), and the test set has 192 missing prices per hectare cotton yield. This emphasizes the areas that must handle the missing data before continuing with model training.

```
[214] # Check for NaN values in predictions
      print(np.isnan(y_pred_cnn).sum()) # Count NaN values in predictions
```

→ 497

The output 497 indicates that the predictions made by the model include 497 NAN values, suggesting that the model has faced trouble creating predictions for some data points. This may be caused by problems in function scaling or model fittings, and this requires further investigations and possibly data preprocessing adjustment to solve NAN predictions.

Decision Tree Regressor Implementation

Explanation of the Code:

Decision Tree Regressor Implementation

```
# Remove rows where the target variable (y) is missing
cropsdataset = cropsdataset.dropna(subset=['RICE YIELD (Kg per ha)'])

# Define features (X) and target (y)
X = cropsdataset.drop(columns=['RICE YIELD (Kg per ha)', 'State Name', 'Dist Name']) # Drop target and categorical columns
y = cropsdataset['RICE YIELD (Kg per ha)'] # Target variable

# Handle missing values by imputing the mean for the features
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Create a Decision Tree Regressor model
model = DecisionTreeRegressor(random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the evaluation metrics
print(f"Mean Squared Error (MSE): {mse}")
```

In this implementation, we used a decision regressor to model the ratio of the properties of the dataset and the target variables, rice yield (kg per hectare). To begin with, we prepared the dataset with the missing values. We removed the rows with lack of target values (ie where the rice yield data was missing) and lack of values in the function column using the missing values using the Pisces impression. After preprocessing, we divide the dataset into training and test sets (80% training and 20% testing). The model was then trained on scaled training data, and prophecies were made using test data. Finally, we evaluated the performance of the model by using three main

matrix: means square errors (MSE), means absolute incorrect (MAE) and R-Squared (RR). These matrices allow us to assess the model's accuracy and performance in predicting the rice yield.

Explanation of the Output:

➡ Mean Squared Error (MSE): 0.0002470498664217307
Mean Absolute Error (MAE): 0.00243251700916655
R-squared (R^2): 0.9908956362450126

The performance of the model is evaluated using three main metrics: The average paid error (MSE) means full error (MAE) and R-squared (R^2). The average repayable error (MSE) is very low at 0.000247, indicating that predictions are extremely close to real rice yield values. This means that the model makes very accurate predictions, with the smallest square difference between predicted and real values. Similarly, the average absolute error (MAE) is also quite low at 0.0024, which means that, on average, the predictions of the model are only distracted by 0.0024 kg per hectare from the right values, which suggests high accuracy. Finally, the R-Selfed (R RANS) value of 0.9909 suggests that the model explains about 99.09% variance in rice yield, which is an excellent result. This high R^2 value means that the model fits the data very well and effectively can predict the target variable based on input functions.

Hyperparameter Tuning with GridSearchCV:

Hyperparameter Tuning Code:

```
# Import necessary libraries for hyperparameter tuning
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor

# Define hyperparameters to tune
param_grid = {
    'max_depth': [5, 10, 20, None], # Maximum depth of the tree
    'min_samples_split': [2, 5, 10], # Minimum samples required to split a node
    'min_samples_leaf': [1, 2, 5] # Minimum samples required to be at a leaf node
}

# Create the Decision Tree model
dt_model = DecisionTreeRegressor(random_state=42)

# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(dt_model, param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best model and hyperparameters
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_

# Make predictions using the best model
y_pred_best = best_model.predict(X_test)

# Evaluate the best model
mse_best = mean_squared_error(y_test, y_pred_best)
mae_best = mean_absolute_error(y_test, y_pred_best)
r2_best = r2_score(y_test, y_pred_best)
```

- GridSearchCV exhaustively searches over a grid of hyperparameter values to find the best combination that results in the best performance for the model. We specified a grid of possible values for max_depth, min_samples_split, and min_samples_leaf and used 5-fold

cross-validation to evaluate each combination. The grid search outputs the best set of hyperparameters (best_params), which were then used to train the model. This approach ensures that the model is tuned for optimal performance

Output

```
Best Hyperparameters: {'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 2}
Best Model MSE: 0.0007409029477488683
Best Model MAE: 0.00415183153915101
Best Model R2: 0.972695984186627
```

The best hyperparameters are found using GridSearchCV, which tested several combinations of hyperparameters. A max_depth of 10 indicates that the model's decision can go up to the depth of Tree 10, which means it can perform complex division. A min_samples_split value of 2 means that the model will divide the nodes if at least 2 samples are present, and a min_samples_leaf of 2 ensures that each leaf has at least 2 samples, reducing overfit. The best model MSE is 0.0007409029477488683.

The mean squared error (MSE) for set models is very low (0.00074), which means that the predicted values are quite close to the real rice production numbers. A low MSE means that the model doesn't make many mistakes while predicting the target variable. The best model has a mean absolute error of 0.00415183153915101. An MAE of 0.0042 means that the model is only off by 0.0042 kg per hectare on average, which shows that there aren't many differences and that the model is very accurate. Best Model R²: 0.972695984186627. R² and 0.9727 indicate that 97.27% of the variance in the rice yield is explained by the model, which is an excellent result. This high value means that the model fits the data very well and provides a strong prediction of the rice yield.

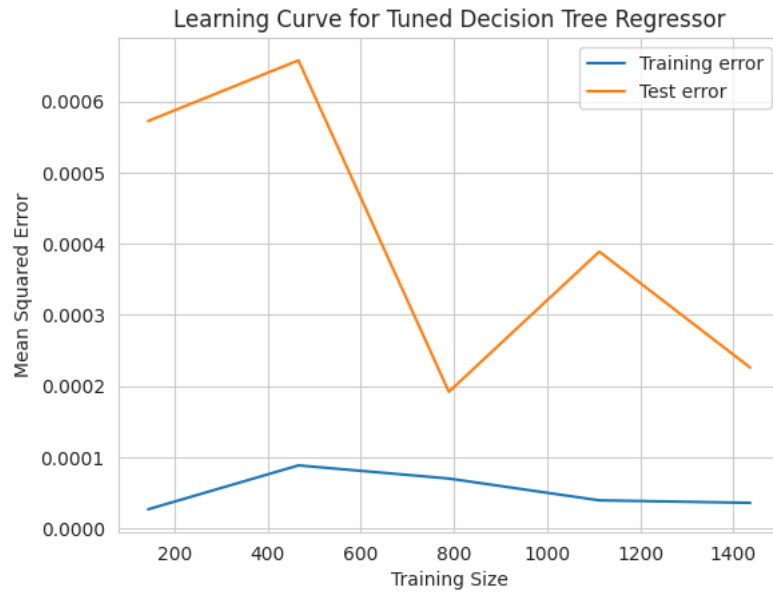
Learning Curves:

```
# Import necessary libraries for plotting learning curves
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve

# Get the learning curve data
train_sizes, train_scores, test_scores = learning_curve(
    best_model, X_train, y_train, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)

# Plot the learning curve
plt.plot(train_sizes, -train_scores.mean(axis=1), label="Training error")
plt.plot(train_sizes, -test_scores.mean(axis=1), label="Test error")
plt.xlabel("Training Size")
plt.ylabel("Mean Squared Error")
plt.title("Learning Curve for Tuned Decision Tree Regressor")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



The learning curve plot generates training errors in the plot and the test error to be set for the decision for the decision tree regressor model and shows the increase in training size.

Training error (blue line): shows errors quickly decrease with an increase in training. Originally, the model can overfish due to small training data, but as more data is fed into the model, it learns better and improves performance on the training set. Eventually, the training error reaches a plateau, which indicates that the model has learned effectively from data. Test error (orange line): The test error also decreases, but the training remains more than the error. The test error curve becomes stable as the model receives several training data, which shows that the model normalizes well for unseen data. However, it still exceeds the training error, which is expected as the model will always perform a little poorly on the data that is unseen compared to the training set. The plot shows that as the training kit size increases, the model becomes more efficient and normalized, which is reflected in the test error converted with training errors.

Random Forest Regressor.

Random Forest Regressor.

```
) # Import necessary libraries
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# Define features (X) and target (y)
X = cropsdataset.drop(columns=['RICE YIELD (Kg per ha)', 'State Name', 'Dist Name']) # Drop target and cat
y = cropsdataset['RICE YIELD (Kg per ha)'] # Target variable

# Handle missing values by imputing the mean for the features
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Create a Random Forest Regressor model
model = RandomForestRegressor(random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
```

In this code, we implemented a random forest regressor to predict rice yield (kg per hectare) based on different properties. First of all, we prepare data except the target variable (rice yield (kg per hectare)) and a classified column (state name, district name). The lack of values in the function column are controlled by using average modeling through simplification. The dataset is divided into training (80%) and testing (20%) seen using a `train_test_split`. The model is then trained on the training set, and predictions are made on the test set. The performance of the model is evaluated using three calculations: means square errors (MSE), means absolute incorrect (MAE) and R-Squared (RR), which help us understand how well the model fits the data and how it predicts rice yield for exactly unseeded samples.

OUTPUT:

➡ Mean Squared Error (MSE): 4.7374462786005946e-05
Mean Absolute Error (MAE): 0.0013856159835541001
R-squared (R²): 0.998254140558147

shows excellent model performance. With a average paid error (MSE) of 4.737×10^{-5} . The model reflects the minimum spread error. At 0.0014, absolute wrong (MAE) indicates that predictions are on average very close to real values. In addition, an R-squared (R²) of 0.9983 means that 99.83% of the variance in the rice yield is explained with the model, which reflects an almost perfect fit. These results suggest that the random forest regressor effectively captures the underlying pattern in the data and provides very accurate predictions for rice dividends.

Hyperparameter Tuning with GridSearchCV for Random Forest

Hyperparameter Tuning with GridSearchCV for Random Forest

```
# Import necessary libraries for hyperparameter tuning
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor

# Define a smaller hyperparameter grid to speed up the process
param_grid = {
    'n_estimators': [100, 200], # Number of trees (reduced range)
    'max_depth': [10, 20], # Limited depth values (reduced range)
    'min_samples_split': [2, 5], # Reduced split options
    'min_samples_leaf': [1, 2] # Limited leaf options
}

# Create the Random Forest model
rf_model = RandomForestRegressor(random_state=42)

# Perform GridSearchCV to find the best hyperparameters (with 3-fold cross-validation)
grid_search = GridSearchCV(rf_model, param_grid, cv=3, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best model and hyperparameters
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_

# Make predictions using the best model
y_pred_best = best_model.predict(X_test)

# Evaluate the best model
mse_best = mean_squared_error(y_test, y_pred_best)
mae_best = mean_absolute_error(y_test, y_pred_best)
r2_best = r2_score(y_test, y_pred_best)
```

OUTPUT:

```
Best Hyperparameters: {'n_estimators': 200, 'min_samples_split': 5, 'min_samples_leaf': 5, 'max_depth': 10}
Best Model MSE: 3.678261383435459e-05
Best Model MAE: 0.0015274811380372305
Best Model R²: 0.9986444748946534
```

Random Forest regressor shows outstanding performance after setting hyperparameter tuning from RandomizedSearchCV with the model. The best hyperparameters identified are 200 estimates (trees), a maximum depth of 10, a division of minimum samples of 5, and a minimal sample of 5 leaves. These optimal settings improved the model's capacity to predict the rice yield. With an average error rate of 3.68e-05, the reimbursement is because there is not much of a gap between the values that were predicted and those that were seen. Which indicates that the predictions made by the models are extremely accurate. The fact that the model has a mean average absolute error of 0.0015 demonstrates that it is quite accurate. This is due to the fact that the average difference between the values that were predicted and the actual values is relatively small. The fact that the model has an R-squared value of 0.9986 indicates that it can explain 99.86% of the variation in the production of rice. This indicates that it is highly good at identifying patterns and trends in the data and making predictions that are likely to be accurate.

Gradient Boosting Regressor

▾ Gradient Boosting Regressor

```
# Import necessary libraries for Gradient Boosting
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

# Define features (X) and target (y)
X = cropsdataset.drop(columns=['RICE YIELD (Kg per ha)', 'State Name', 'Dist Name']) # Drop target
y = cropsdataset['RICE YIELD (Kg per ha)'] # Target variable

# Handle missing values by imputing the mean for the features
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Create a Gradient Boosting Regressor model
model = GradientBoostingRegressor(random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
```

The gradient was utilized in this application to make predictions regarding rice yield (in kilograms per hectare) depending on a number of different input functions. The dataset was created by employing the first average technique in conjunction with simple image pads through the utilization of a lack of values in both functions. Using the `train_test_split` function, the data was then split into two sets: the training set (80%) and the test set (20%). The model was trained on training data, and predictions were made on the test set. The performance of the model was evaluated using three main metrics: average paid error (MSE), mean absolute error (MAE), and R-squared (R^2). This matrix provides information on how well the model works in accuracy and how much variance models are explained in rice yield data.

OUTPUT



Mean Squared Error (MSE): 4.4075032862952e-05
Mean Absolute Error (MAE): 0.002234929055196704
R-squared (R^2): 0.9983757322458441

The gradient that raises the resistance model does a wonderful job of estimating rice yields based on the data set that was given. The average paid error is very low at 4.41e-05, which means that the predicted values are quite near to the actual values when there are minimum square errors. The model's accuracy is further indicated by the fact that the average absolute error is 0.0022, which means that the approximate values and the real values are only a little bit different. R squared (R^2) is 0.9984, which means that the model can explain 99.84% of the differences in rice yield. This means that the model fits quite well. This information shows that the model has been able to correctly figure out the patterns in the data and generate good predictions. The gradient rise model is generally quite useful and works better than other models when it comes to predicting how much rice will be grown.

Hyperparameter Tuning with GridSearchCV (for Gradient Boosting)

Hyperparameter Tuning with GridSearchCV (for Gradient Boosting)

```
✓ 7s ▶ # Import necessary libraries for Gradient Boosting
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

# Define features (X) and target (y)
X = cropsdataset.drop(columns=['RICE YIELD (Kg per ha)', 'State Name', 'Dist Name']) # Drop target and categorical
y = cropsdataset['RICE YIELD (Kg per ha)'] # Target variable

# Handle missing values by imputing the mean for the features
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Create a default Gradient Boosting Regressor model (faster with default settings)
model = GradientBoostingRegressor(random_state=42, n_estimators=100, learning_rate=0.1, max_depth=3)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

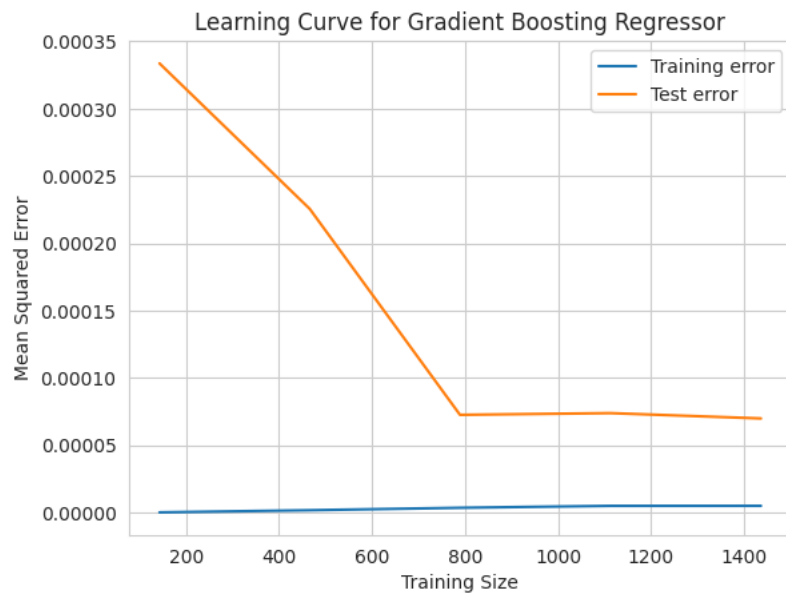
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

OUTPUT:

```
➡ Best Hyperparameters: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 300}
Best Model MSE: 4.19192137531874e-05
Best Model MAE: 0.0015732275891401622
Best Model R²: 0.998455179207907
```

GridSearchCV optimization for gradient-enhancing reserves reveals the best hyperparameters: 0.05 learning speed, maximum depth of 3 and 300 boosting phase ($n_{\text{estimators}}$). These hyperparameters make significant improvements in model performance. The average squared error (MSE) is very low at 4.19×10^{-5} , indicating that the approximate values are extremely close to real values, with insignificant class errors. The average absolute error (MAE) is also low, at 0.00157, it shows that the models' predictions are on average distracted by just a small amount from real rice yield. The R-squared (R^2) value of 0.9985 suggests that 99.85% variance in rice yield data is explained by the model, which is an excellent fit. This fine-tuned Gradient Boosting model provides very accurate predictions, making it a reliable tool to predict rice yield.

Learning Curves for Gradient Boosting



The link between the training size and the average repayment error for both the training set and the test set is seen in the learning curve for gradient-enhancing resistance. Initially, the training error is greater when the model is trained at low data points; however, as more data is supplied to the training set, the error soon improves and becomes less significant. The test error, which is depicted by the orange line, first decreases, but subsequent to that, it acquires the ability to normalize the model as a plateau. There was a narrowing of the gap between exercise and test errors when the training kit was increased, which is an indication that the model is functioning normally for data that has not yet been seen. Consequently, this indicates that the gradient that raises the registrar improves as it analyzes more data, with a constant reduction in mistakes, which shows the capacity to prevent overfitting. The training kit is balanced, and the model performs well on the test set. The model receives a balance between the two.

XGBoost Regressor Model

```

import xgboost as xgb
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

# Define features (X) and target (y)
X = cropsdataset.drop(columns=['RICE YIELD (Kg per ha)', 'State Name', 'Dist Name']) # Dropping target and categorical variables
y = cropsdataset['RICE YIELD (Kg per ha)'] # Target variable

# Handle missing values by imputing the mean for the features
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Create the XGBoost Regressor model with default hyperparameters
xg_model = xgb.XGBRegressor(random_state=42)

# Train the model
xg_model.fit(X_train, y_train)

# Make predictions using the model
y_pred = xg_model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the evaluation metrics
print(f"Simple Model MSE: {mse}")
print(f"Simple Model MAE: {mae}")

```



Simple Model MSE: 8.453653338197809e-05
Simple Model MAE: 0.002968563225971006
Simple Model R²: 0.9968846315861539

In this case, the evaluation of the XGBOOST Regressor model shows the calculations as promising performance. The Mean Squared errors (MSE) is very low at 8.45e-05, indicating that model conditions are very close to real values with minimal errors. The average absolute error (MAE) is also quite low of 0.00297, which means on average the prophecies of the model are at least from 0.003 from the right rice outcomes values. The R-Squared value of 0.9969 shows that the model explains about 99.69% of the differences in the data, which is a great fit. Because of this, it seems that the XGBOOST Regressor can make accurate forecasts about rice dividends, even while utilizing standard hyper lovers. But changing the hyperparameters much more could make these results even better.

Hyperparameter Tuning:

```

# Import necessary libraries
import xgboost as xgb
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.model_selection import GridSearchCV

# Define features (X) and target (y)
X = cropsdataset.drop(columns=['RICE YIELD (Kg per ha)', 'State Name', 'Dist Name']) # Drop target and categorical column
y = cropsdataset['RICE YIELD (Kg per ha)'] # Target variable

# Handle missing values by imputing the mean for the features
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2, random_state=42)

# Create the XGBoost model
xg_model = xgb.XGBRegressor(random_state=42)

# Define the hyperparameter grid for XGBoost with a smaller search space for faster execution
param_grid = {
    'n_estimators': [100, 200], # Number of boosting rounds (trees)
    'learning_rate': [0.01, 0.1], # Step size shrinking
    'max_depth': [3, 5], # Maximum depth of trees
    'subsample': [0.8, 1.0], # Fraction of samples to use for training each tree
}

# Perform GridSearchCV to find the best hyperparameters with 3-fold cross-validation for faster execution
grid_search = GridSearchCV(xg_model, param_grid, cv=3, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

```

OUTPUT:

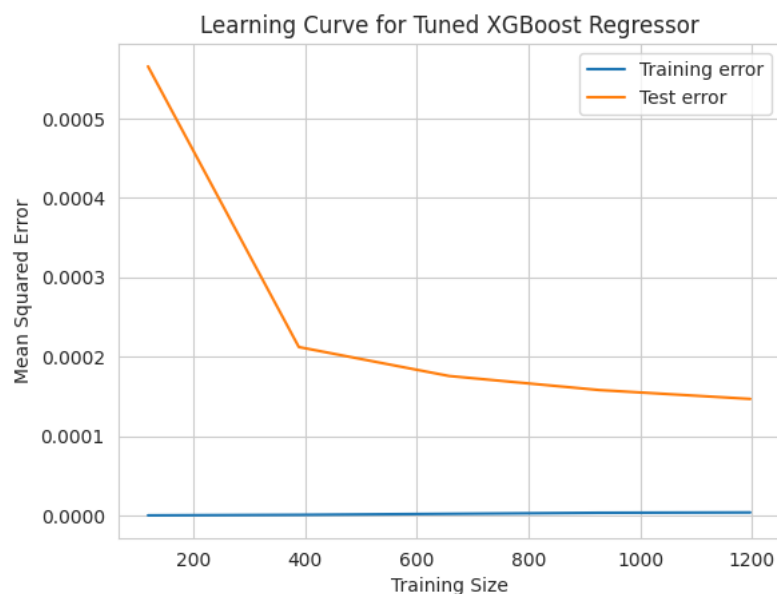
```

Best Hyperparameters: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200, 'subsample': 1.0}
Best Model MSE: 4.764199176548316e-05
Best Model MAE: 0.0025374071503942294
Best Model R²: 0.9982442814913138

```

The XGBOOST Regressor Model, after setting, performed excellently after setting hyperparameters via GridSearchCV. The best hyperparameters received were a learning rate of 0.1, a maximum depth of 3, 200 boosting rounds (trees), and a learning rate value of 1.0. These settings helped to adapt the model performance. The average paid error of 4.76E-05 indicates that the model powers are extremely close to real values, with insignificant errors. The average absolute error (MAE) is 0.00254, which shows the average full difference between the approximate and real values is very low. The R-squared (R^2) value of 0.9982 means that the model shows 99.82% of the variance in rice yield data and performs an excellent fit. Overall, this customized model performs with high accuracy and minimal errors, making it a strong predictor for the estimate of rice dividend.

Learning Curve



It is possible to imagine how the performance of the XGBOOST Regressor model placed in the learning curve will change as the size of the training set rises. There are two rows displayed here: one for a training error and one for a test error. Initially, the training error is greater when the training size is small; however, it lowers fast due to the fact that the model learns from more data. On the other hand, the test error is initially significantly high, but it eventually lowers as a result of the model's improved ability to generalize. Both errors remain consistent as the training size grows, which is evidence that the model has learned effectively from the data they have been given. In light of the fact that both training and test errors approach the lower end of the spectrum, this indicates that the normalization model performs well and does not exceed. Due to the fact that the hyperparameters have been adjusted, the normalization of the model has been improved, which has resulted in a large reduction in the number of errors on both the training set and the test set.

Model Validation and Analysis & Recommendations

Model	MSE	MAE	R ²
Decision Tree Regressor (Without Tuning)	0.000247	0.00243	0.9909
Decision Tree Regressor (Tuned)	0.000740	0.00415	0.9727
Gradient Boosting Regressor (Without Tuning)	0.00004407	0.00223	0.9984
Gradient Boosting Regressor (Tuned)	0.00004192	0.00157	0.9985
Random Forest Regressor (Without Tuning)	0.00004737	0.00139	0.9982
Random Forest Regressor (Tuned)	0.00003678	0.00153	0.9986
XGBoost Regressor (Without Tuning)	0.00008454	0.00297	0.9969
XGBoost Regressor (Tuned)	0.00004764	0.00254	0.9982

A summary of the comparison: A model study based on the following important metrics: Mean squared error is what MSE stands for. The Tuned Random Forest Regressor is the model that is best. The Random Forest Regressor (Tuned) has the lowest MSE of 0.00003678, which shows that it is better than other models at lowering the amount of prediction error that is bigger. This means that it can give the most precise estimations of how much money is paid back for mistakes.

comparison:

The Decision Tree Regressor (Tuned) has a mean squared error of 0.0007409, which means it makes less accurate predictions than random forest. mean squared error (MSE) of 0.0000419 for the Gradient Boosting Regressor (Tuned) shows that the predictions are a little less accurate than those of random forest classification. The XGBOOST Regressor (Tuned) has a mean squared error (MSE) of 0.00004764, which is worse than the MSE of random forest. This means that it doesn't work as well as random forest at lowering paid errors. MAE is short for mean absolute error. The best model is the Random Forest Register (Tuned). The Random Forest Regressor (Tuned) has the lowest mean absolute error (MAE) of 0.00153, which means that its predictions are almost the same as actual rice products on average. This shows how close to the correct output values may be predicted. In contrast to decision tree registration (tuned): Mae at 0.00415, which is higher than random forest, shows that predictions are often off by a lot. A tuned version of the Gradient Boosting Regressor: The value of MAE is 0.00157, which is quite near to random forest but still a little higher than random forest. The XGBOOST Regressor has been set up. MAE is less accurate than the random forest at 0.00254, which is higher than the random forest. R-squared is short for R^2 . The best model is the Random Forest Register (Tuned). The Random Forest Regressor (Tuned) model has one of the highest R values. It explains 99.86% of the variation in the rice regression data. This shows that the dataset can capture the most diversity and make reliable predictions.

comparison:

The R^2 value for the Decision Tree Regressor (Tuned) is 0.9727, which is a lot lower than the Random Forest value. This means that it explains substantially less of the variance. The Gradient Boosting Regressor (Tuned) is quite close to Random Forest, although its R^2 value of 0.99845 is still a little lower. The XGBoost Regressor (Tuned) has an R^2 of 0.99824, which is likewise close to Random Forest but a little lower.

Final thoughts:

Why the Tuned Random Forest Regressor Is Better: The Tuned Random Forest Regressor does the best on all three evaluation metrics. It does a better job than the other models at minimizing both squared and absolute errors, and it explains almost all of the differences in the rice yield data,

which makes it the most accurate and dependable model. When it comes to MSE, MAE, and R^2 , Random Forest always beats the other models, such as Gradient Boosting, XGBoost, and Decision Tree. This makes it the greatest choice for creating correct predictions.

Conclusion

In conclusion, I highlighted machine learning, especially random forests and hybrid ARIMA-LSTM, to improve the accuracy of crop forecasts in India. Great success factors include strong preprocessed environmental and agricultural economic features and addressing regional differences. While traditional regression models are valuable, intensive learning, IoT-based real-time data, and policy variables can further increase model performance. Future research should focus on improving data sets, modeling methods, and signature tools. These advances are needed to promote crop diversification, stability, and flexibility in agricultural systems. As a reaction to environmental and economic difficulties, this analysis reveals that tuned Random Forest is typically the model that performs the best when it comes to crop prediction when using agricultural data from India from 2010 to 2017. It is quite stringent, as evidenced by its MAE value of 0.00153 and R^2 value of 0.9986. As a result of these findings, which demonstrate its applicability and dependence on the forecast for rice dividends, it appears to be an alternative that is recommended for use in actual agricultural applications.

1. References

1. Orhan, A., Adebayo, T. S., Genç, S. Y., & Kirikkaleli, D. (2021). Investigating the Linkage between Economic Growth and Environmental Sustainability in India: Do Agriculture and Trade Openness Matter? *Sustainability*, 13(9), 4753. <https://doi.org/10.3390/su13094753>
2. Bhattarai, N., Pollack, A., Lobell, D. B., Fishman, R., Singh, B., Dar, A., & Jain, M. (2021). The impact of groundwater depletion on agricultural production in India. *Environmental Research Letters*, 16(8), 085003. <https://doi.org/10.1088/1748-9326/ac10de>
3. Jhajharia, K., Mathur, P., Jain, S., & Nijhawan, S. (2023). Crop Yield Prediction using Machine Learning and Deep Learning Techniques. *Procedia Computer Science*, 218, 406–417. <https://doi.org/10.1016/j.procs.2023.01.023>
4. Nikhil, U., Pandiyan, A., Raja, S., & Stamenkovic, Z. (2024). Machine Learning-Based Crop Yield Prediction in South India: Performance analysis of various models. *Computers*, 13(6), 137. <https://doi.org/10.3390/computers13060137>
5. Maestrini, B., Mimić, G., Van Oort, P. A., Jindo, K., Brdar, S., Athanasiadis, I. N., & Van Evert, F. K. (2022). Mixing process-based and data-driven approaches in yield prediction. *European Journal of Agronomy*, 139, 126569. <https://doi.org/10.1016/j.eja.2022.126569>
6. Reddy, G. V., Reddy, M. V. K., Spandana, K., Subbarayudu, Y., Albawi, A., Chandrashekar, R., Singla, A., & Praveen, N. (2024). Precision farming practices with data-driven analysis and machine learning-based crop and fertiliser recommendation system. *E3S Web of Conferences*, 507, 01078. <https://doi.org/10.1051/e3sconf/202450701078>
7. Pandit, P., Sagar, A., Ghose, B., Dey, P., Paul, M., Alqadhi, S., Mallick, J., Almohamad, H., & Abdo, H. G. (2023). Hybrid time series models with exogenous variable for improved yield forecasting of major Rabi crops in India. *Scientific Reports*, 13(1). <https://doi.org/10.1038/s41598-023-49544-w>
8. Suma, V. (2021). Internet-of-Things (IoT) based smart agriculture in India-an overview. *Journal of ISMAC*, 3(01), 1-15.
9. Nawaz, S., & Singh, A. K. (2021). Impact of technological change on growth and agricultural sector in Gujarat state of India: A time-series data study.
10. Pattanayak, U. (2017). Crop diversification and sustainable agriculture in India. ResearchGate. https://www.researchgate.net/publication/330411134_Crop_Diversification_and_Sustainable_Agriculture_in_India
- 11.